

疎結合化の追求

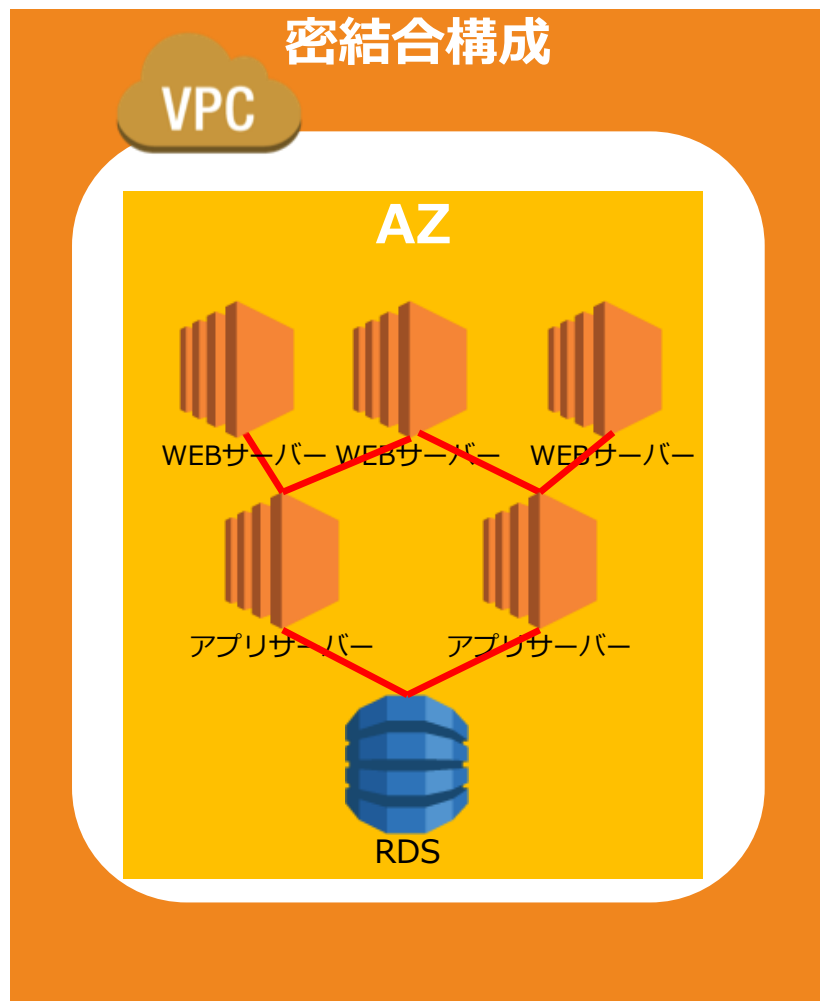
コンポーネントの疎結合

コンポーネント間の相互依存を減らした構成とすることで、1つのコンポーネント変更や障害の影響を削減する



密結合の問題

密結合したアーキテクチャは障害や修正に弱く、不具合が発生しやすい

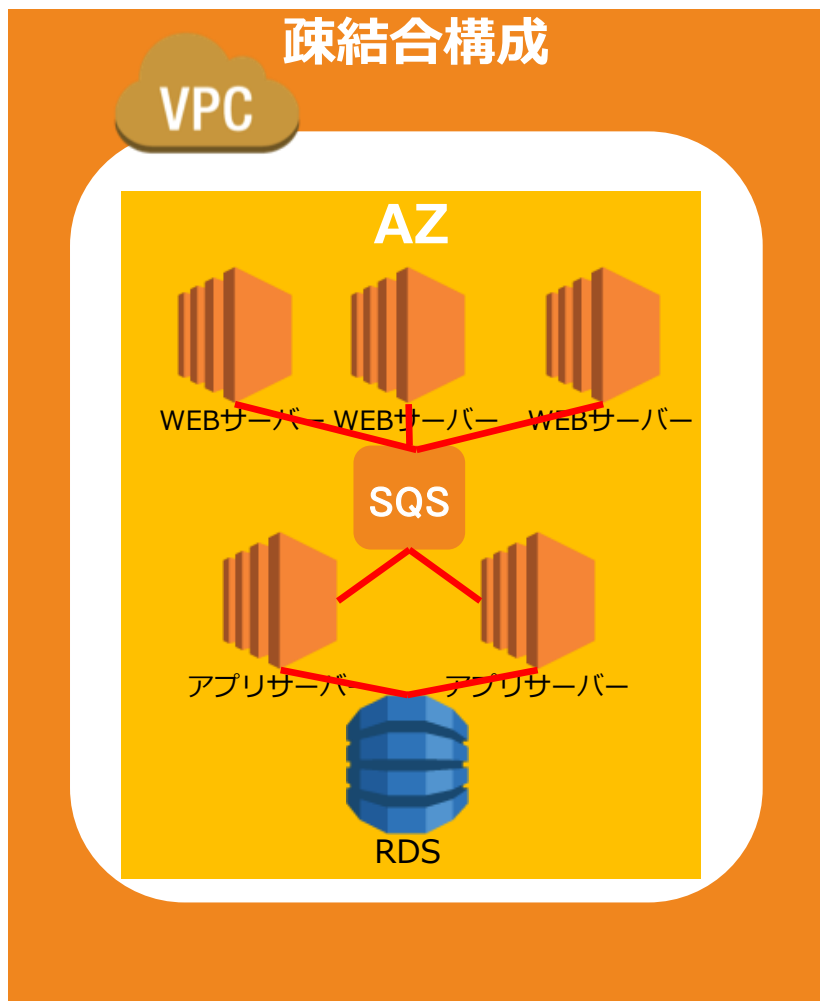


【密結合はデメリットが多い】

- ❑ 1インスタンスの障害の影響が全体に及びやすい
- ❑ 1つの修正対応で他インスタンスへの影響を多く考慮しなければならない
- ❑ 負荷対応やスケーリングなども容易にできない
- ❑ システム構成の追加・変更が難しい

疎結合化のメリット

ELBやAPIなどを利用して結合点を削減したり、メッセージ結合にすることで影響を減らすことができる



【疎結合化のメリット】

- 耐障害性が強まる
- 負荷対応やスケーリングなどが容易
- システム構成の追加・変更が容易

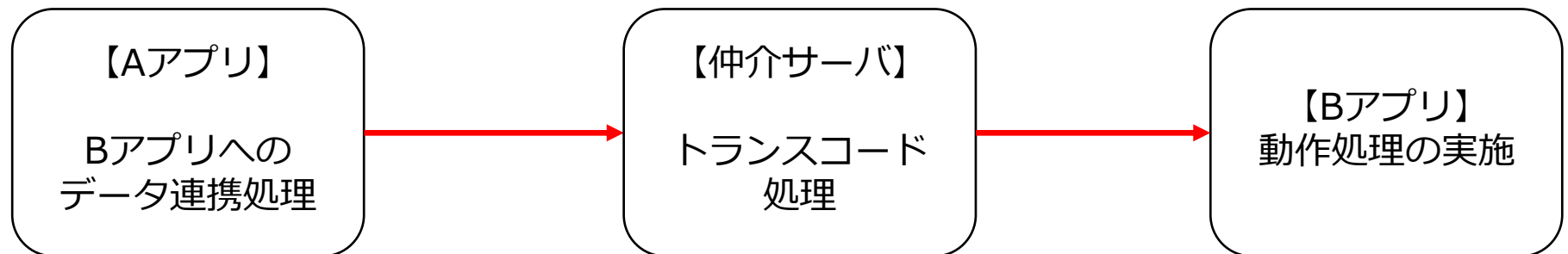
疎結合向けサービス

サーバレス化をするサービスやメッセージング処理をするサービスを利用して疎結合化する

ELB	サーバー間のトラフィック調整と連携をELBを起点に結ぶことで疎結合化を実現
SQS	SQSのキューイングによる通信でインスタンス間連携を結ぶことで疎結合化を実現
SNS	SNSのアプリケーション間通信でインスタンス間連携を結ぶことで疎結合化を実現
Lambda	インスタンスではなくLambdaによるトリガー処理で連携することで疎結合化を実現

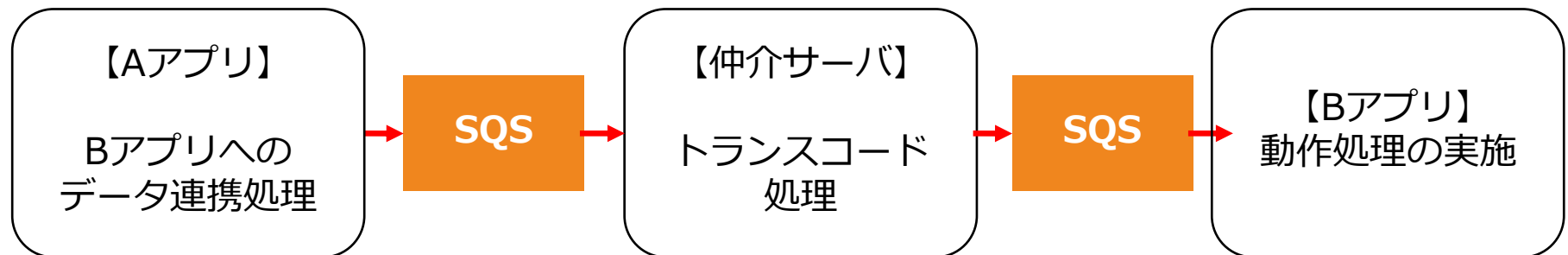
通信系サービスによる疎結合

サーバー間の連携を直接結んで連携すると密結合となってしまうが、



通信系サービスによる疎結合

サーバー間の連携をメッセージング処理で結ぶことで疎結合化を実現する



疎結合設計

疎結合設計にはサーバレス／キューイング通信／マネージド型サービスの利用などを駆使した設計を行う

密結合タイプの設計パターン

- ユーザー認証・管理をバックエンドサーバーで処理
- 通常のEC2インスタンスでアプリケーションを構成
- アプリケーション間では直接通信
- 静的ウェブシステムをEC2インスタンスEBSに保存

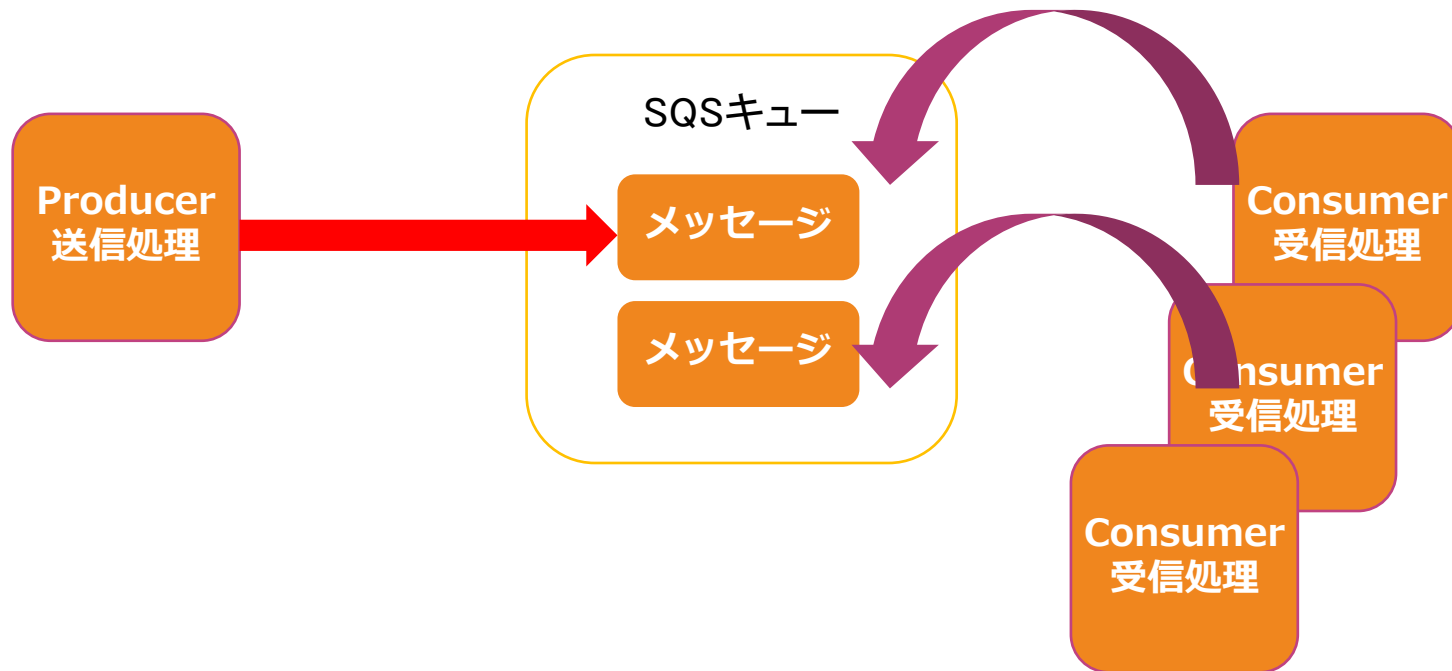
疎結合化向けの設計パターン

- ユーザー認証・管理をIAMなどのマネージド型サービスを利用
- なるべくLambdaなどサーバレスでアプリケーションを構成
- アプリケーション間ではSQSなどMQ通信で連携
- 静的ウェブシステムをVPC外部のS3に保存
- トランザクション単位でマイクロサービスを設計する

Amazon SQSの概要

SQSとは何か？

タスクのトリガーとなるキューを複数管理することで、ワークロードの並列実行を実現するキューイングサービス



SQSの選択

SQSはポーリング処理型のキューイングサービスで、タスクの並行実施などに利用される。

Amazon SNS	完全マネージド型 pub/sub メッセージングを実施するサービス。メール通知やプッシュ通知による連携処理に利用する。
Amazon SQS	完全マネージド型のキューイングサービス ポーリング処理によるタスクの並列実施に利用する。
Amazon SES	Eメール機能を可能にするサービス。アプリケーション上にEメール送受信機能を実装する際に利用する。安全、グローバル、大規模に E メールを送信が可能になる。
Amazon MQ	JMS、NMS、AMQP、STOMP、MQTT、WebSocket などの業界標準 API やメッセージング用プロトコルを使用するApache ActiveMQ 向けのマネージド型メッセージブローカーサービス
Amazon Kinesis Data Streams	5MBまでの容量のあるデータをストリーム処理することが可能で、シャード単位でデータを順番通りに送信することができる。

SQSの特徴

AWSの様々なサービスと連携して通知可能で、疎結合アーキテクチャに実現する。

【基本的な機能】

- ❑ 単一発行メッセージをキューとして利用
- ❑ ポーリング処理型のキューイングサービス
- ❑ 標準キューはメッセージ通信順番は保証されないが、FIFOキューは順番を保証する。
- ❑ 優先キューは他のキューよりも優先的に処理させることが可能
- ❑ メッセージ保持期間の間はメッセージを保持するが、超過するとメッセージを削除する。
- ❑ 発行したメッセージは取り消し不可
- ❑ 配信ポリシーによるキューの再試行を実施する

SQSの特徴

SQSはキューを発行・蓄積して、ポーリング処理を管理する。



SQSの特徴

SQSはキューを発行・蓄積して、ポーリング処理を管理する。



SQSの特徴

SQSはキューを発行・蓄積して、ポーリング処理を管理する。



SQSの特徴

SQSはキューを発行・蓄積して、ポーリング処理を管理する。



SQSキューの特徴

無制限にメッセージを利用可能だが、メッセージ保持期間をうまく設定することが必要

メッセージの制約

メッセージ数は無制限に利用可能
メッセージサイズは最大256KB
ただし、拡張クライアントライブラリーを利用すると2GBまでのメッセージのやり取りが可能となる。
コンシューマー側では一度に10件までメッセージを受信可能

メッセージの保持期間

SQSのキューメッセージは保持期間の間は保存される。
デフォルト4日間（最小60秒～最大14日で設定可能）
アプリケーション上でメッセージを削除する処理を実施しないと、期間を超過するまでキューが滞留してしまう。
DeleteMessageAPIを使ってメッセージを削除しない限り、保持期間中のメッセージは処理後も保持される。

SQSのキュータイプ

SQSでは標準キューとFIFOキューのどちらかを選択して、SQSを初期設定することになる。

標準キュー

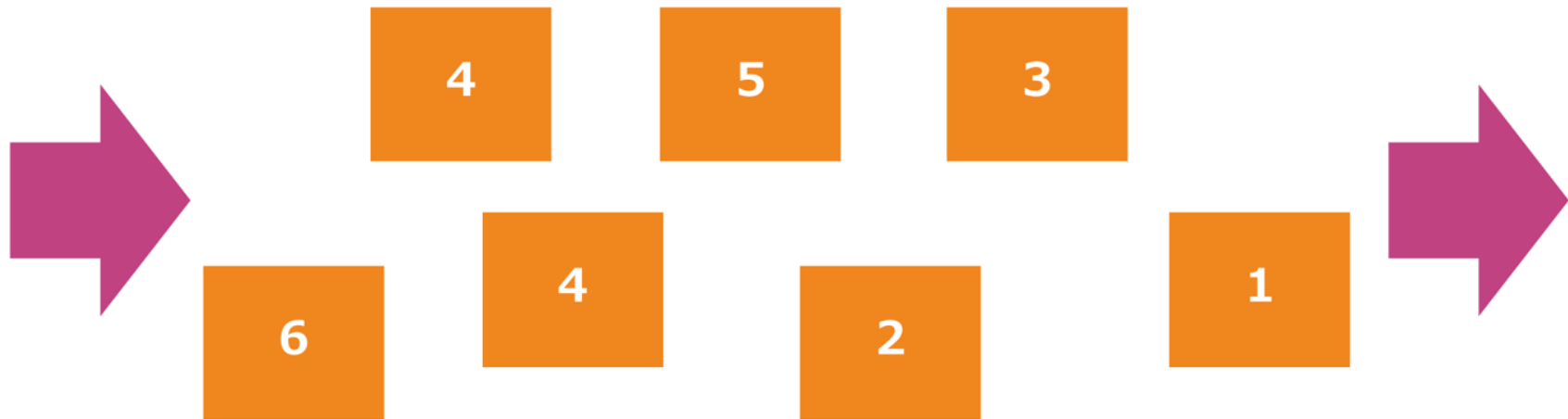
- ✓ メッセージの1つ以上のコピーが順序どおりに配信できないことがある。
- ✓ メッセージが少なくとも1回配信される方式であり、キューには重複が発生する可能性がある。
- ✓ 1秒あたりのトランザクション数はほぼ無制限
- ✓ 標準キューは、アプリケーションが1回以上に順序が正確でなくても配信されれば良いケースで利用する。

FIFOキュー

- ✓ 先入れ先出し方式（FIFO）により配信順番を守る。
- ✓ メッセージが1回だけ配信され、コンシューマがプロセスを処理して削除するまで使用可能なキューの状態を保つため、キューに重複がない。
- ✓ 1秒あたり300トランザクションに制限
- ✓ FIFO キューは、操作やイベントの順序が重要である場合や、重複を許容できないユースケースに利用する。

標準キュー

標準キューは「順番通りの処理」と「1回だけのメッセージング」を“なるべく”実施するキュー方式



FIFOキュー

その名の通り、最初に入ったキューを最初処理する順番を守るキュー方式



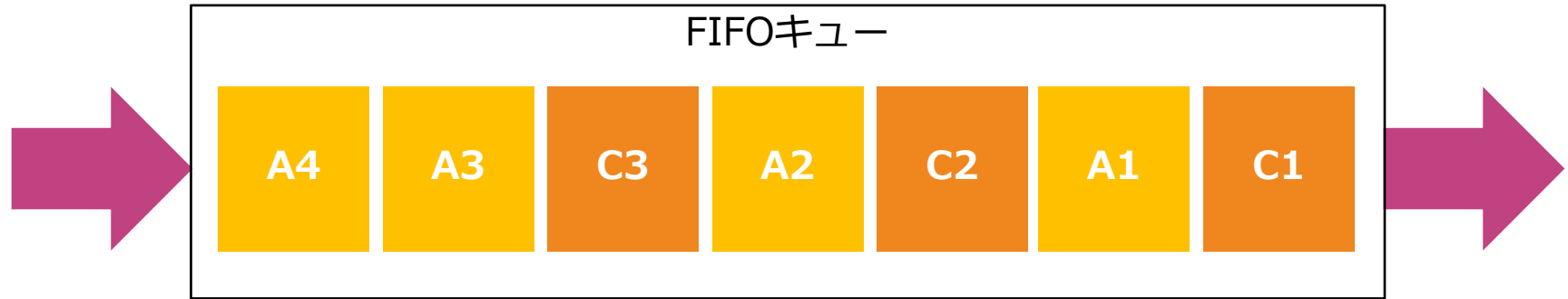
SQSの識別子

SQSではキューを利用する際に様々な機能を利用することが可能。ユースケースに応じて使い分ける必要がある。

キューURL	キューに割り当てられるURL
メッセージID	メッセージに対して割り当てられたID
メッセージグループID	- メッセージグループ ID は 特定のメッセージグループに属するメッセージを指定するタグ - 同じメッセージグループに属するメッセージは、メッセージグループに相対的な厳密な順序で 1 つずつ処理される、 - 単一の FIFO キュー内で複数の順序付きメッセージグループをインターリーブするには、メッセージグループ ID 値を使用する。

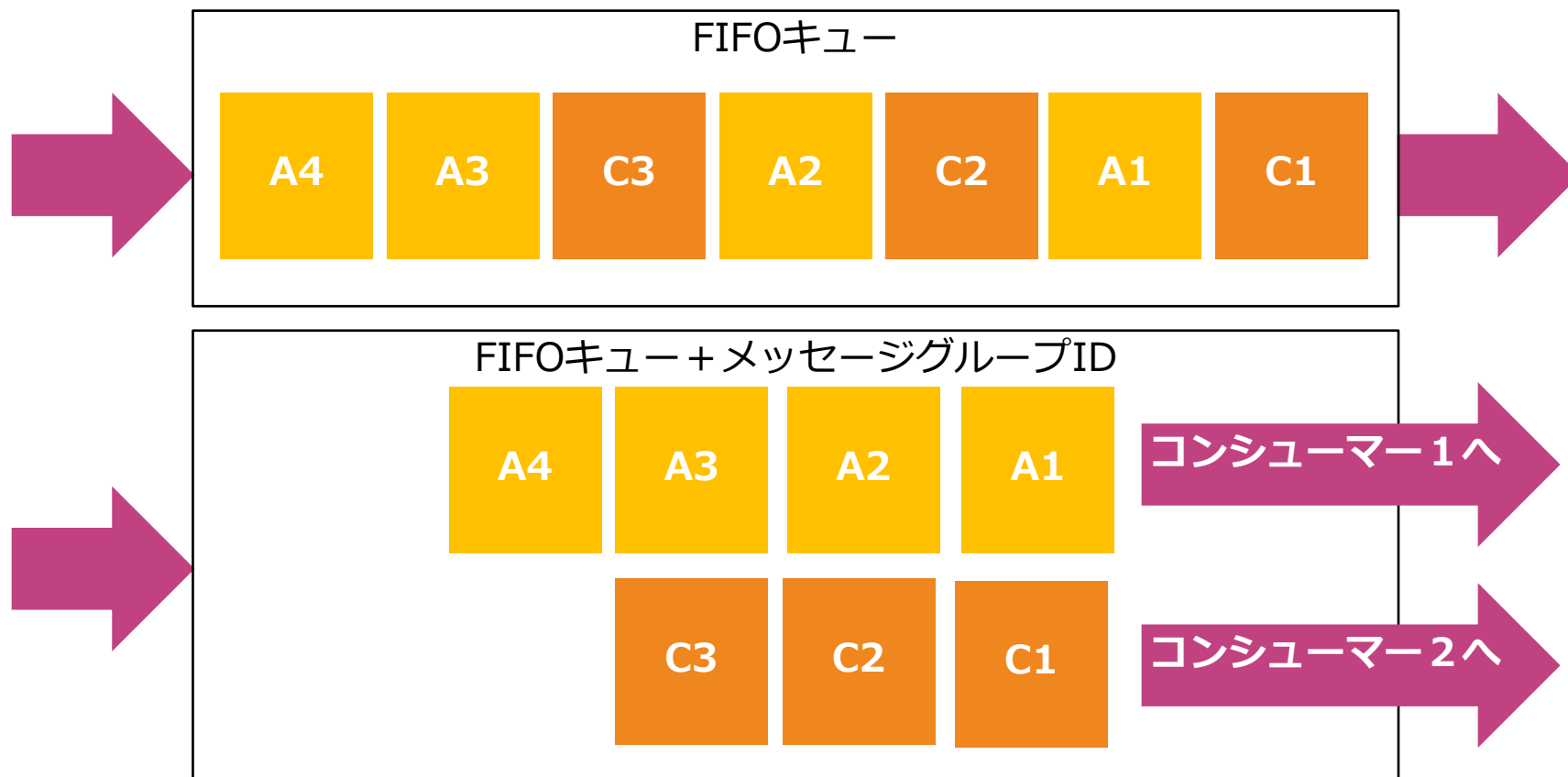
メッセージグループIDによるグループ化

メッセージIDを利用することで同じグループのメッセージはまとめて送信されるようにFIFOキューを調整する。



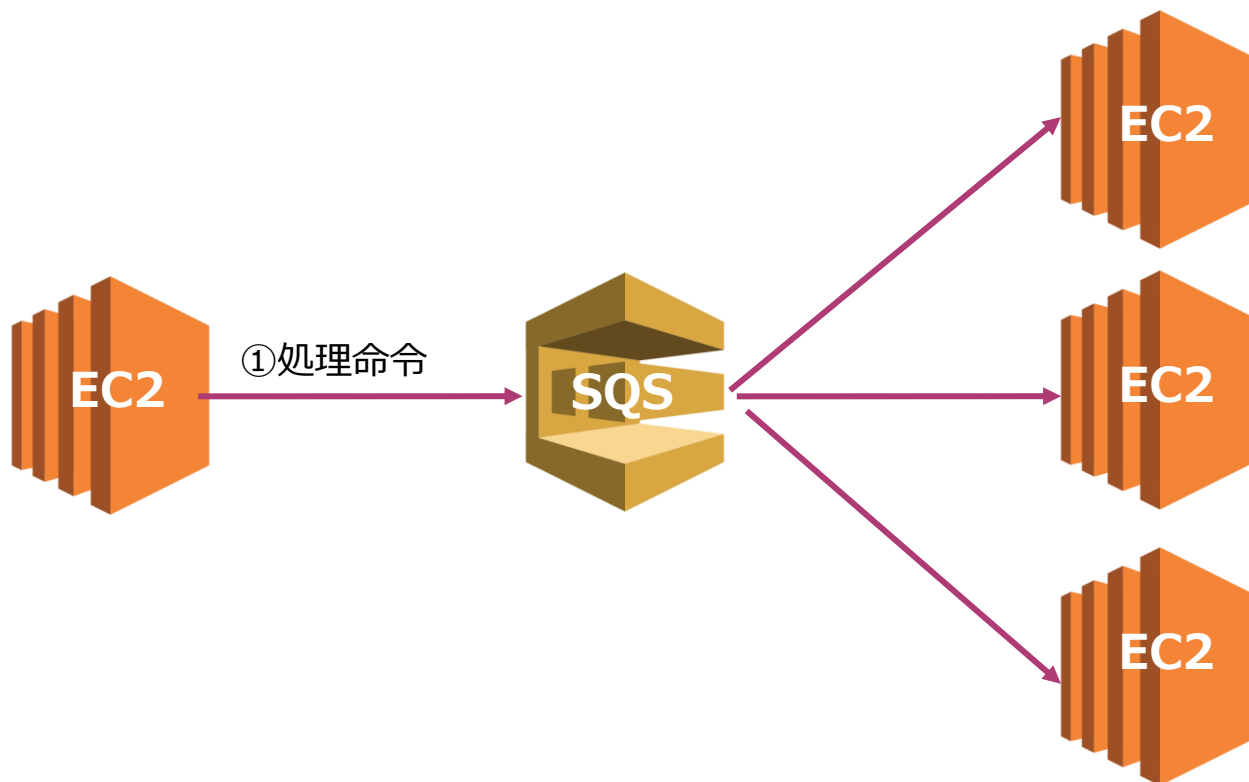
メッセージグループIDによるグループ化

メッセージIDを利用することで同じグループのメッセージはまとめて送信されるようにFIFOキューを調整する。



可視性タイムアウト

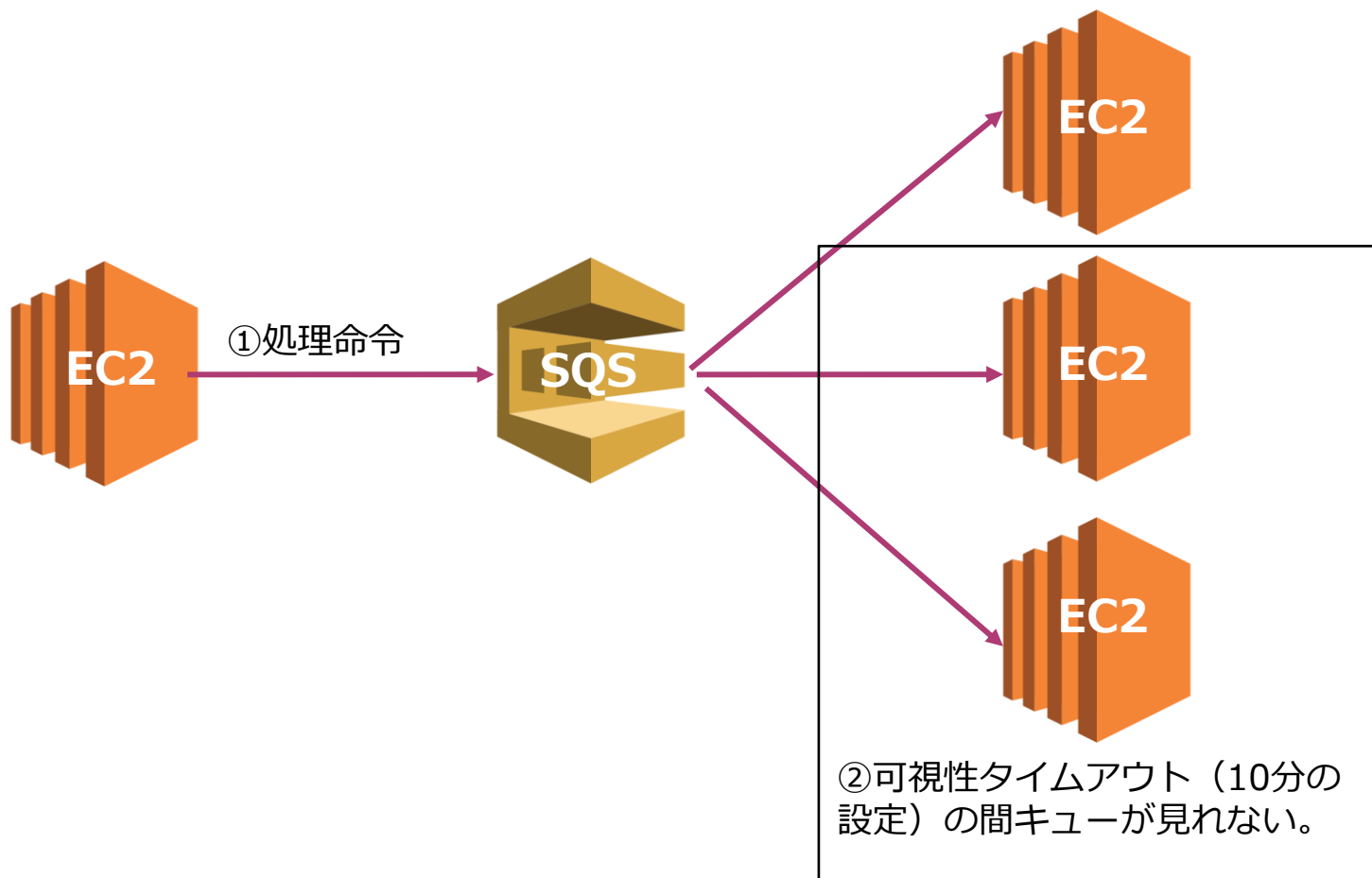
可視性タイムアウトは処理担当のインスタンス以外からは一定時間（30秒～12時間）キューが見えなくなる機能



メッセージが受信された直後は、メッセージはキューに残ったままとなる。他のコンシューマーが同じメッセージを再処理しないように、Amazon SQS は可視性タイムアウトを設定することで、重複処理を防ぐことができる。

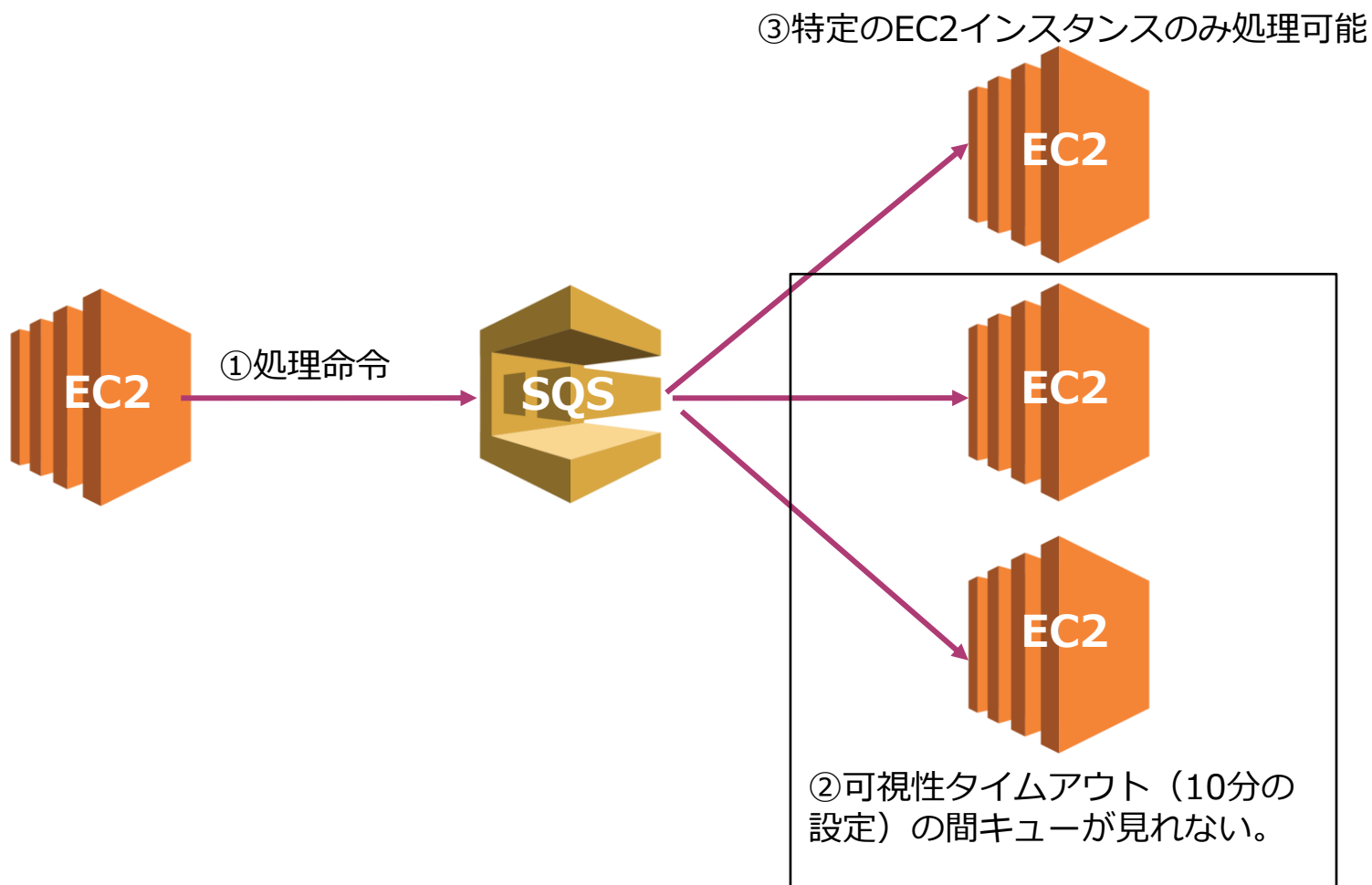
可視性タイムアウト

可視性タイムアウトは処理担当のインスタンス以外からは一定時間（30秒～12時間）キューが見えなくなる機能



可視性タイムアウト

可視性タイムアウトは処理担当のインスタンス以外からは一定時間（30秒～12時間）キューが見えなくなる機能



ポーリングの方式

ポーリング処理の方式でショートポーリングとロングポーリングの2通りがある。

ロングポーリング

問い合わせの結果が空であった場合に、指定したメッセージ受信待機時間はSQSは待機してから応答を返す。
メッセージ受信待機時間は0秒から20秒で設定
空のレスポンス数を削減することができる。

ショートポーリング

キューが空の場合にすぐに空のメッセージが返される

キュータイプ

SQSではキューを利用する際に様々な機能を利用することが可能。ユースケースに応じて使い分ける必要がある。

遅延キュー

キューへの新しいメッセージの配信を数秒間遅延させることができる機能（0秒から15分で設定）

可視性タイムアウトとの違いは、キューが発行された直後から見えなくなること。またキュー全体に効果がある。

キューレベルでのデフォルト設定が可能

DelaySecondsパラメーターを利用してデフォルトを更新可能

優先度付きキュー

キューの処理順序に優先度をつけることができる。

これにより、優先対応があるタスクを最初に処理するようにワークフローを設定できる。

デッドレターキュー

このキューは、正常に処理（消費）できないメッセージを別のキューへと移動させる。

処理不能なキューが蓄積されるのを防ぎつつ、処理できなかった理由を後で解析できる。

キュータイプ

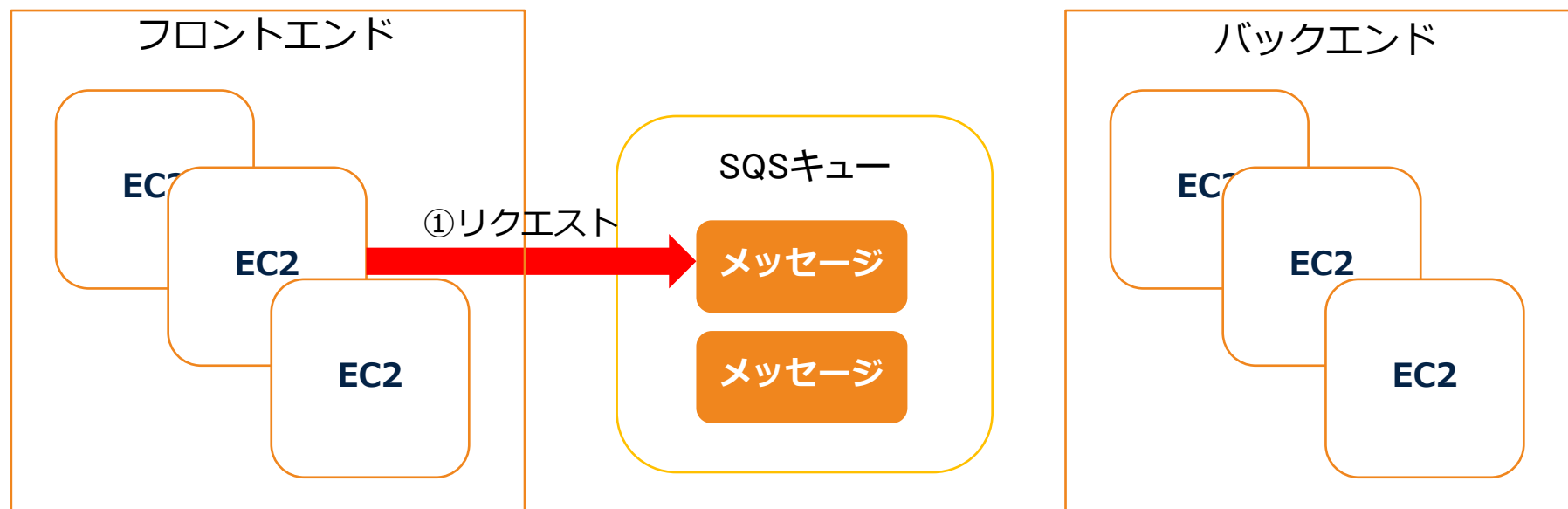
一時キューは高スループットでコスト効率に優れたアプリケーション管理向けに利用されるキュー

一時キュー

- 各一時キューへのトラフィックが少ない場合はAPI 呼び出しが少なくなり、スループットを高める
- 一時キューが使用されなくなると、クライアントは一時キューを自動的にクリーンアップする。
- 特定スレッドまたはプロセスの軽量通信チャネルとして機能
- 追加のコストが発生することなく作成および削除可能
- 仮想キューとの間でメッセージを送受信を実施
- 一時キュークライアントを利用して、リクエスト応答などの一般的なメッセージパターンを開発できる。

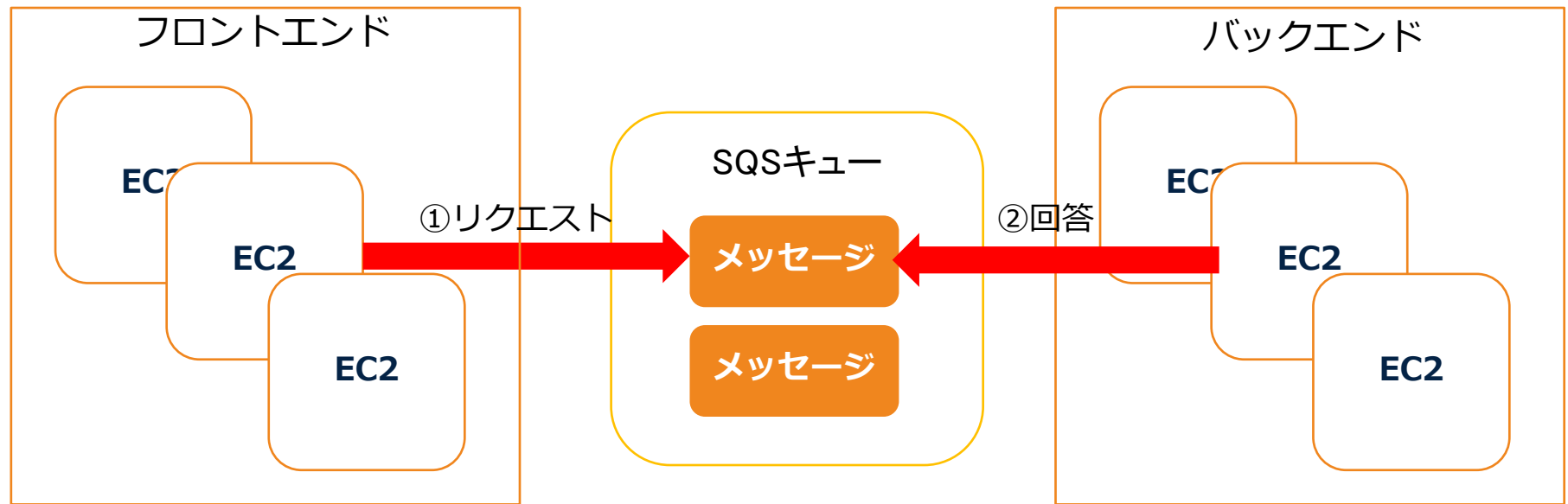
リクエスト/レスポンス構成

一時キュークライアントを利用してリクエスト・レスポンス処理を実装可能



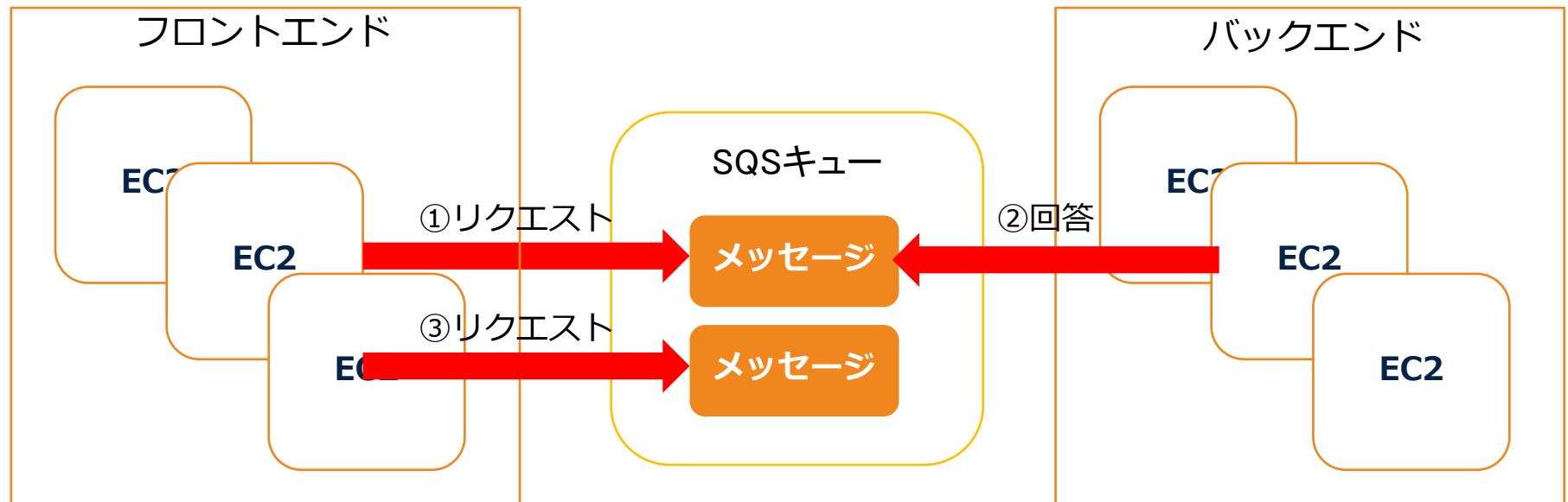
リクエスト/レスポンス構成

一時キュークライアントを利用してリクエスト・レスポンス処理を実装可能



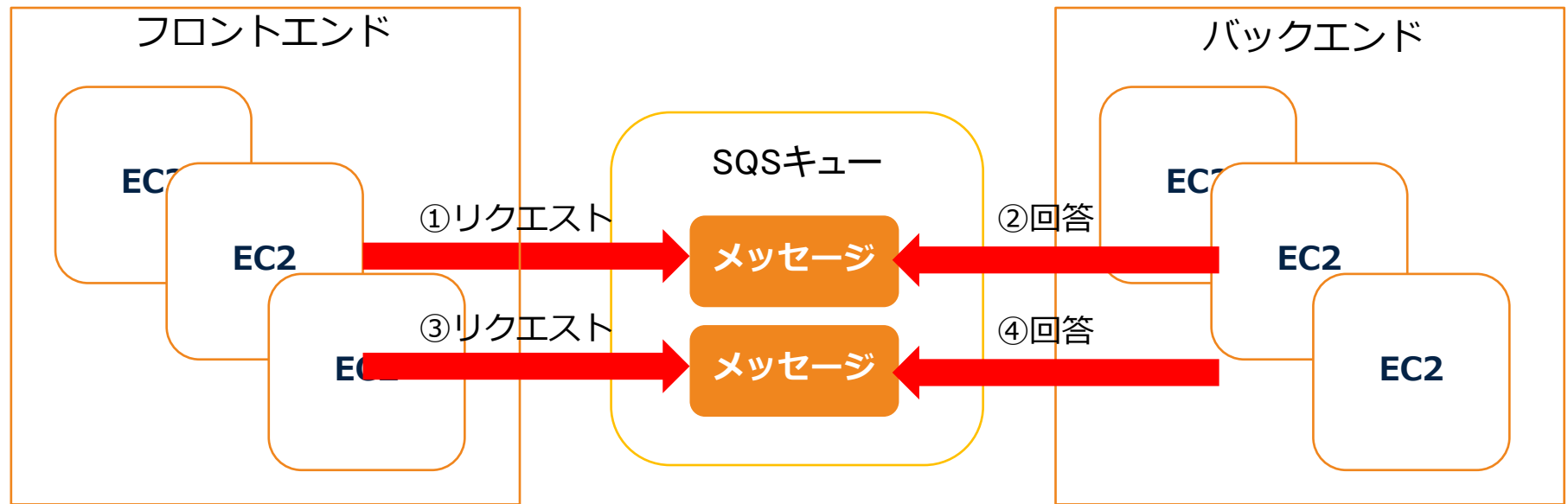
リクエスト/レスポンス構成

一時キュークライアントを利用してリクエスト・レスポンス処理を実装可能



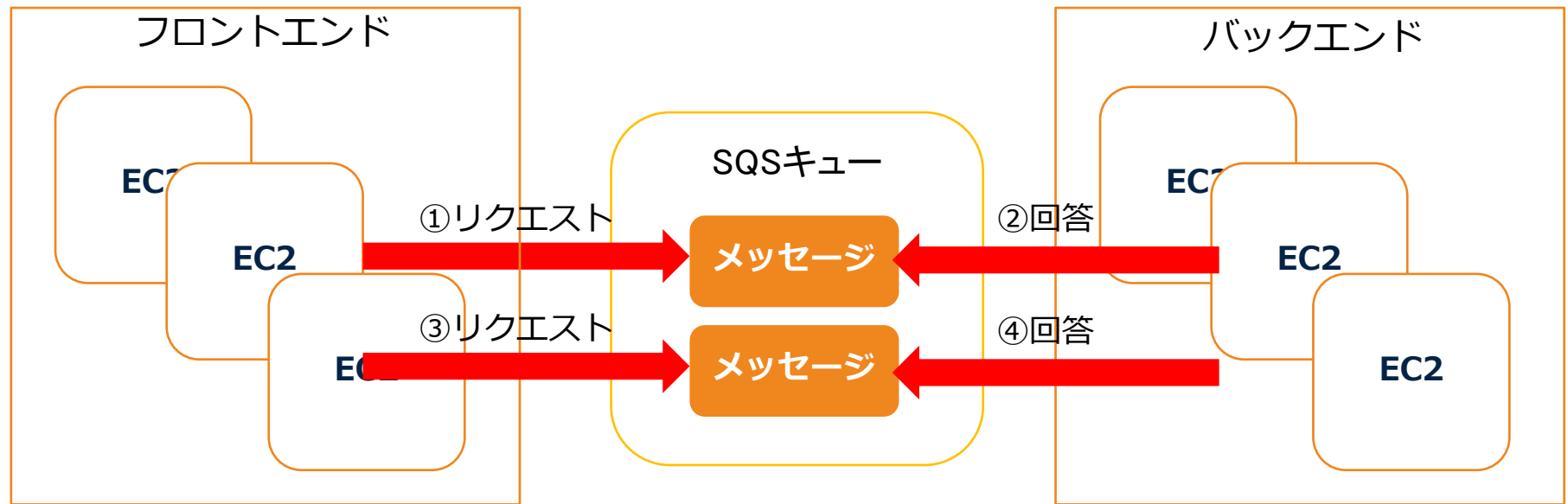
リクエスト/レスポンス構成

一時キュークライアントを利用してリクエスト・レスポンス処理を実装可能



リクエスト/レスポンス構成

一時キュークライアントを利用してリクエスト・レスポンス処理を実装可能



キューの詳細設定

SQSではキューを利用する際に様々な機能を利用することが可能。ユースケースに応じて使い分ける必要がある。

メッセージ 重複排除ID

- 送信されたメッセージの重複排除に使用するトークン
- 同一の重複排除IDが設定されたメッセージをキューへ送っても5分間の間は受け付けられないように設定できる。
- 個別のメッセージグループではなくキュー全体に適用される。
- FIFOのみで利用する

暗号化

- AWS Key Management Service (AWS KMS)を使用して、送信データを暗号化する。

メッセージタイマー

- メッセージタイマーはメッセージが発出された瞬間から、キューに追加されたメッセージが表示されないようにする機能。45秒のタイマーでメッセージを送信すると、キューの最初の45秒間は表示されない。
- 個々のメッセージではなくキュー全体に対して遅延の秒数を設定するには、遅延キューを使用する。
- 個々のメッセージのメッセージタイマー設定はキュー全体よりも優先される。

アクセス管理

SQSポリシーを利用して他アカウントや他リソースからのアクセスをコントロールすることができる。

IAMポリシー

- 一般的なIAMポリシーによるSQSリソースへのアクセス許可を設定する。
- SQSサービスへのアクセスを制御する。

SQSポリシー

- リソースベースポリシーとしてSQSリソース側でアクセス対象を制御するポリシー
- 他のアカウントとSQSキューを共有する許可が可能
- 他のサービスがSQSキューにメッセージを送信することを許可するなどの設定が可能

アクセス管理

SQSポリシーを利用して他アカウントや他リソースからのアクセスをコントロールすることができる。

```
{
  "Version": "2012-10-17",
  "Id": "Queue1_Policy_UUID",
  "Statement": [{
    "Sid": "Queue1_SendMessage",
    "Effect": "Allow",
    "Principal": {
      "AWS": [
        "111122223333"
      ]
    },
    "Action": "sqs:SendMessage",
    "Resource": "arn:aws:sqs:us-east-2:444455556666:queue1"
  }]
}
```

参照: https://docs.aws.amazon.com/ja_jp/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-basic-examples-of-sqs-policies.html

SQSのバッチアクション

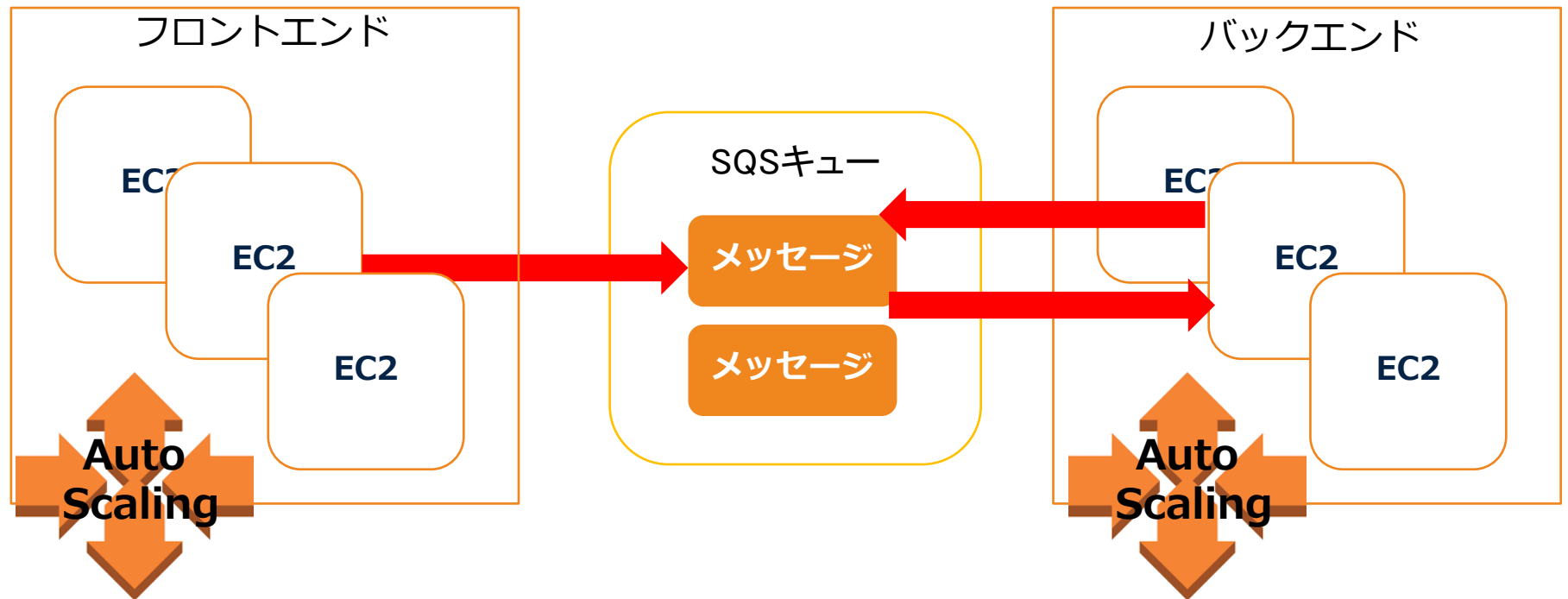
バッチアクションは1 回のアクションで複数のメッセージを操作するなどのバッチ処理が設定可能

Amazon SQS バッチアクションをサポートする
AWS SDK を使用して、バッチ機能を活用できる。

- SendMessageBatch
- DeleteMessageBatch
- ChangeMessageVisibilityBatch

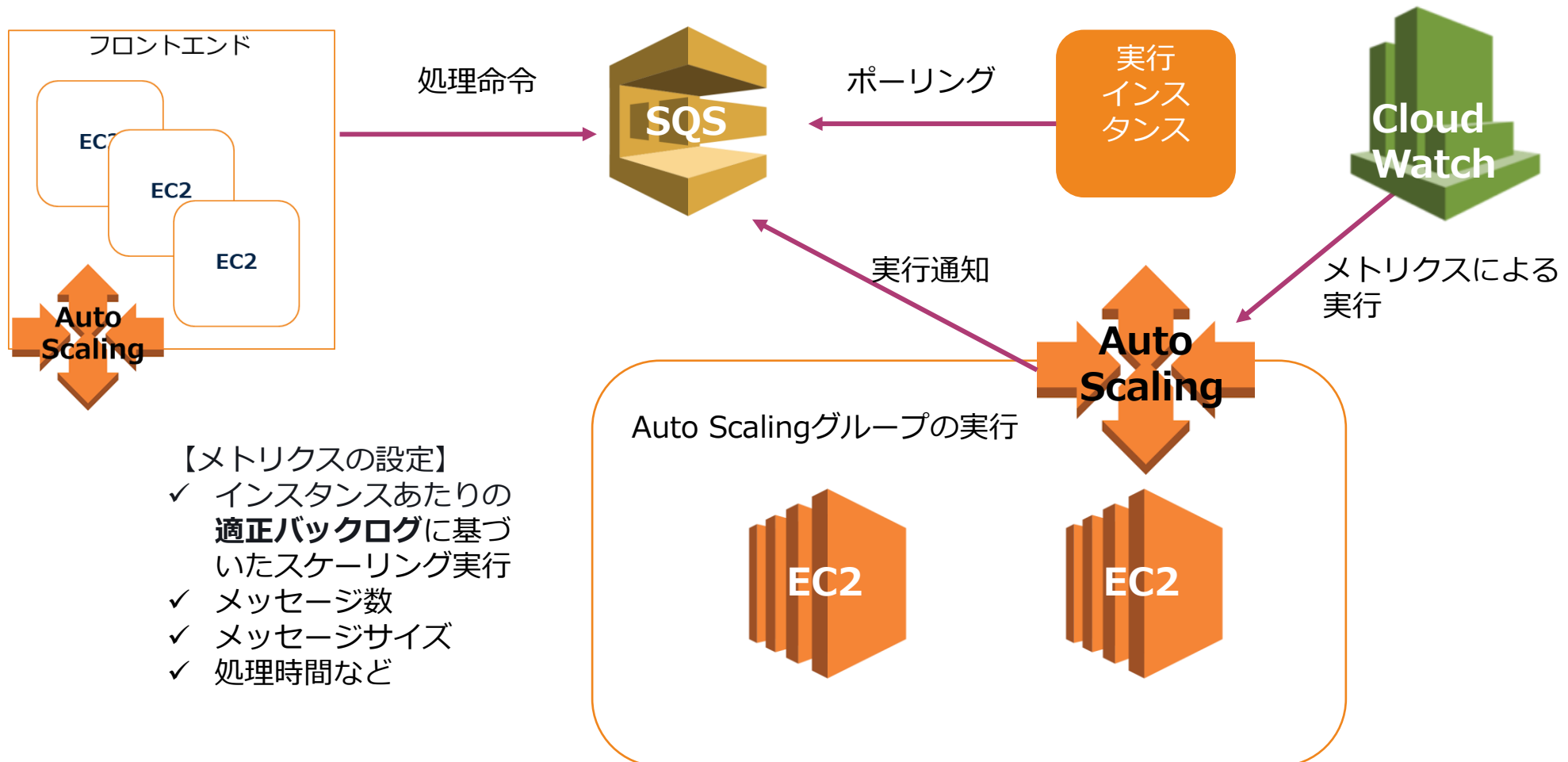
SQSの基本構成

フロントエンドサーバーからキューがトリガーされて、バックエンドの処理サーバーが並列処理するのがSQSの基本構成



SQSとAuto Scalingの構成

SQSとAuto Scalingを構成する際は、CloudWatchメトリクスに基づいてキューの処理量に応じたスケーリングを設定する。

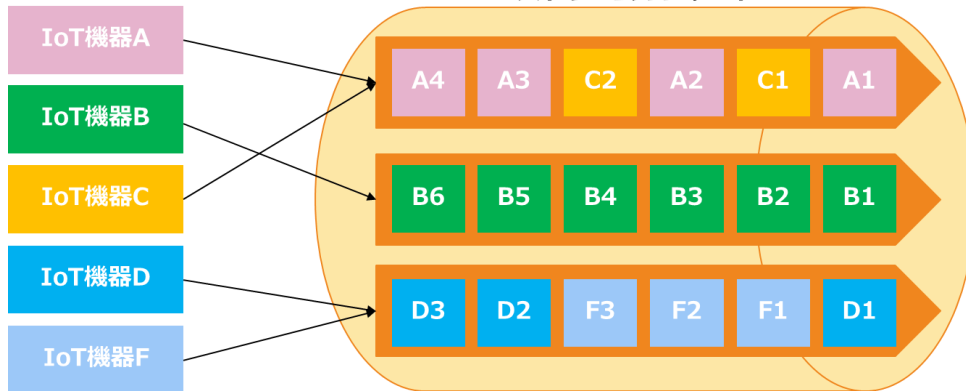


KinesisとSQS

SQSにはコマンド(命令)を投入し、Kinesisにはデータを投入するという使い分けをする。

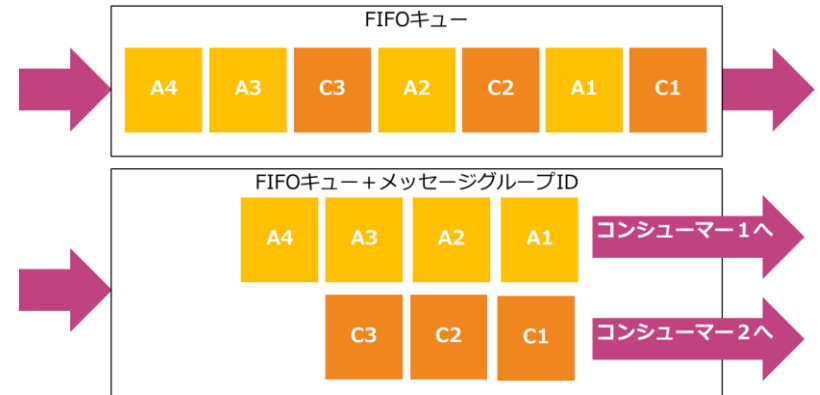
Kinesis

ストリームのシャード



最大 5 MBのデータをメッセージに投入可能

SQS

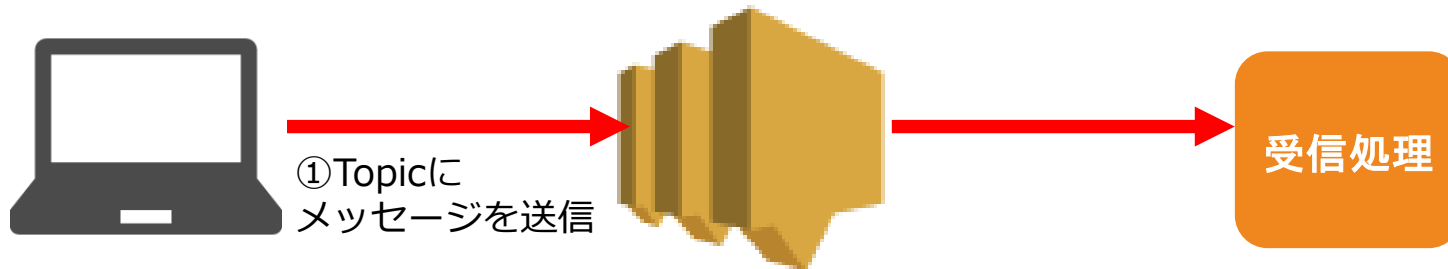


最大256KBのデータをメッセージに投入可能

Amazon SNSの概要

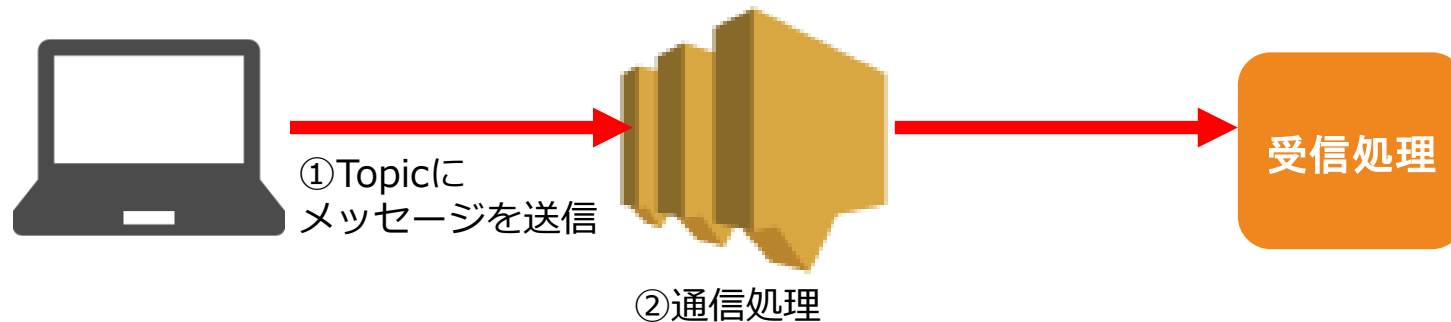
SNSとは何か？

Amazon SNSはフルマネージド型のプッシュ型通知サービスで他のサービスとの非同期通信を可能にする



SNSとは何か？

Amazon SNSはフルマネージド型のプッシュ型通知サービスで他のサービスとの非同期通信を可能にする



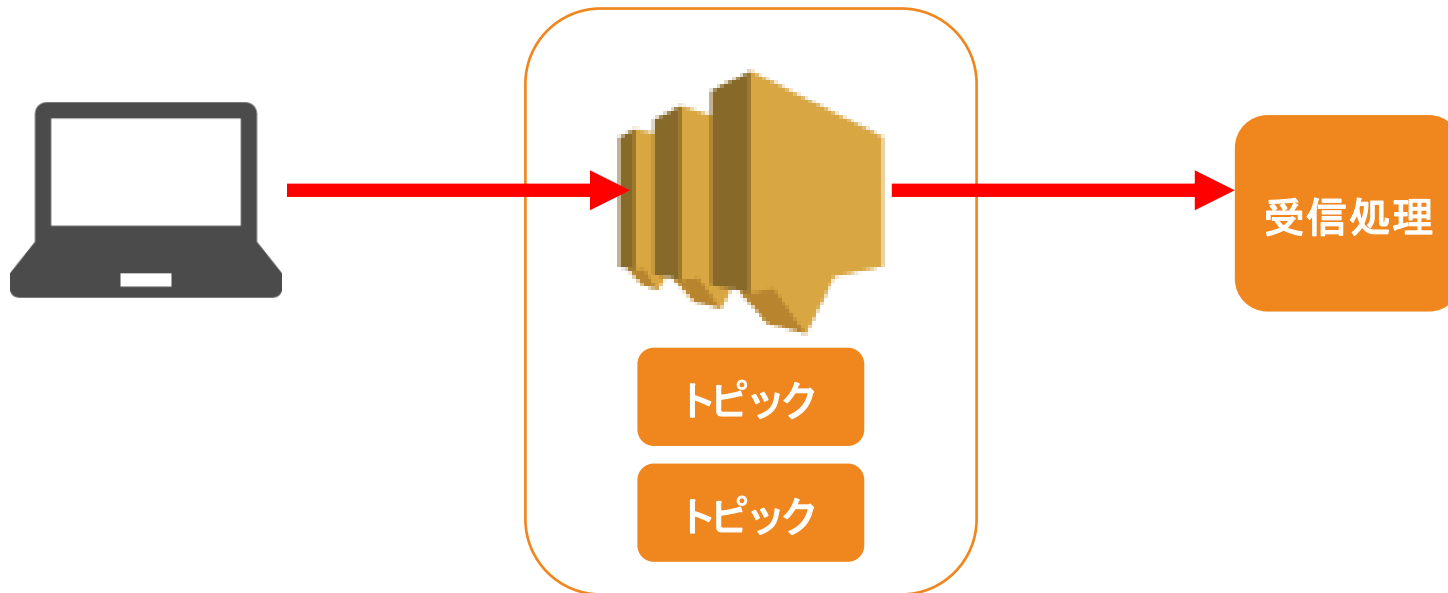
SNSとは何か？

Amazon SNSはフルマネージド型のプッシュ型通知サービスで他のサービスとの非同期通信を可能にする



Amazon Simple Notification Service(SNS)

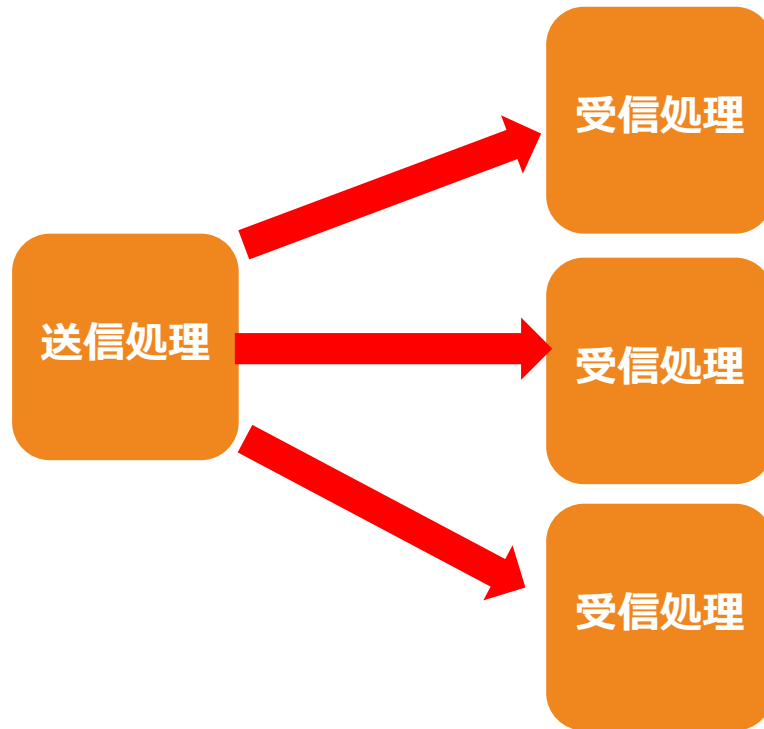
送信側がトピックを作成して受信側をポリシー指定することで
制御された非同期通信を実現する



コンポーネント間の疎結合化

コンポーネント間通信の間にSNSを利用することで、通信処理を効率化することができる。

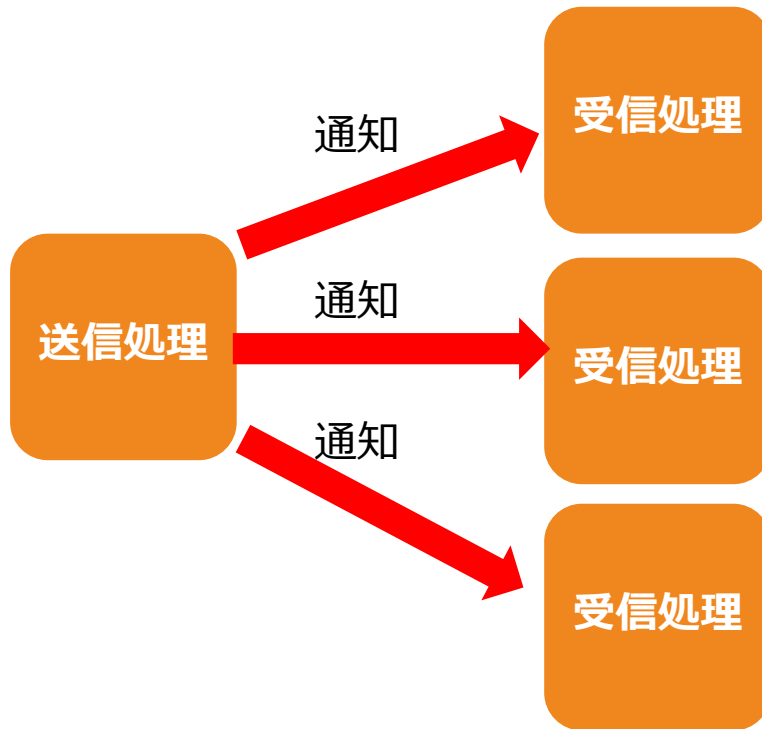
受信先に直接通知を実施する構成



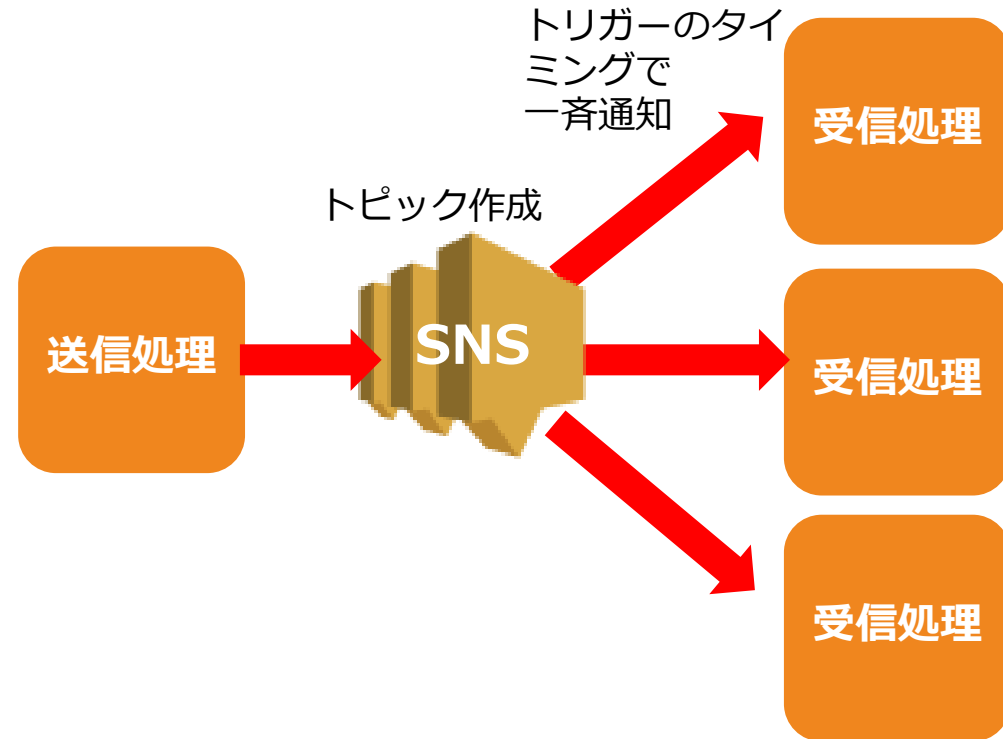
コンポーネント間の疎結合化

コンポーネント間通信の間にSNSを利用することで、通信処理を効率化することができる。

受信先に直接通知を実施する構成



SNSを利用した通知構成



SNSとSQS

SNSとSQSはその処理方式が異なるため利用シーンに応じて使い分ける

SNS

- メッセージは永続ではない
- 配信されたらメッセージは終了
- プッシュ型配信方式
- プロデューサーがトピック発行
- コンシューマーがサブスクライブ
- トリガーのタイミングで通知を実施

SQS

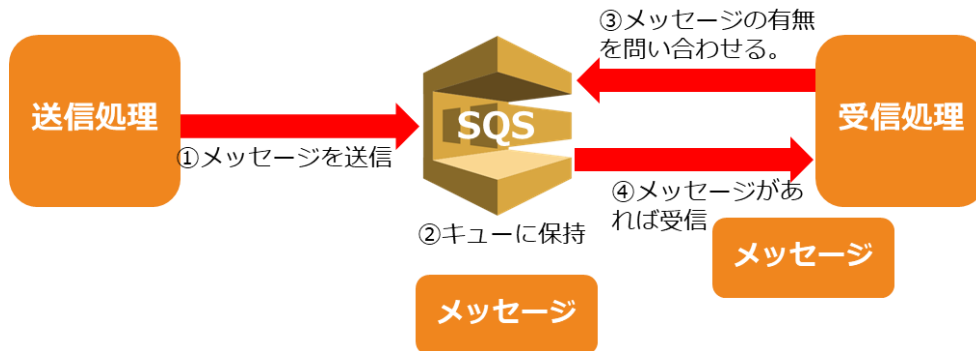
- メッセージは永続性あり
- コンシューマーがメッセージを削除することが必要
- ポーリング型配信方式
- プロデューサーが送信してメッセージをキューに送る
- コンシューマーが自身のタイミングでメッセージを取りに行く

SNSとSQS

コンポーネント間通信の間にSNSを利用することで、通信処理を効率化することができる。

SQSは受信先を気にせずに連携する際に利用

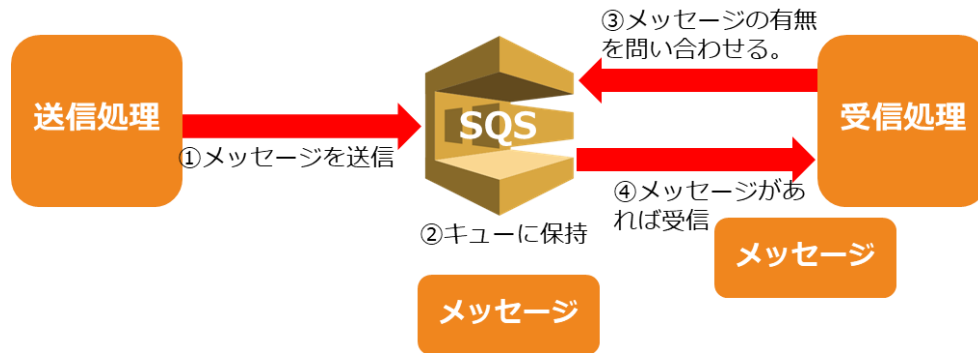
SNSは特定タイミングに通知したいときに利用



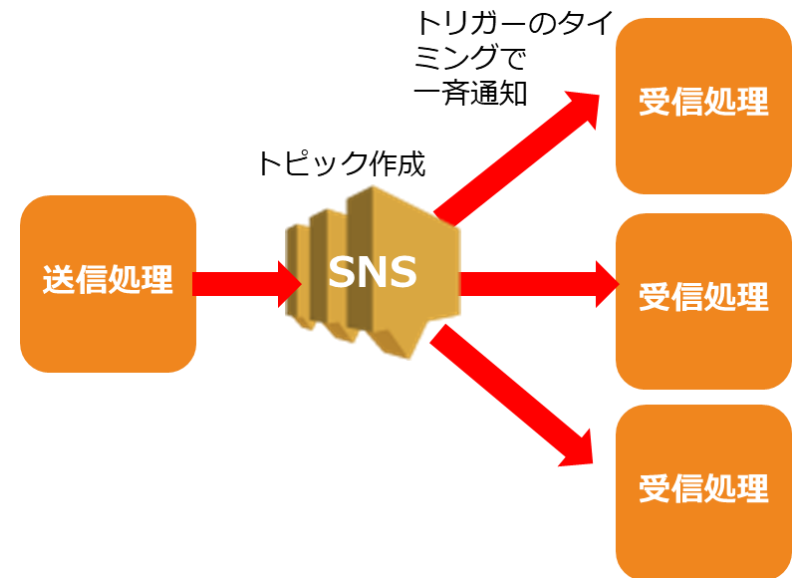
SNSとSQS

コンポーネント間通信の間にSNSを利用することで、通信処理を効率化することができる。

SQSは受信先を気にせずに連携する際に利用



SNSは特定タイミングで通知したいときに利用



SNSの特徴

AWSの様々なサービスと連携して通知可能で、疎結合アーキテクチャに利用できる

- 単一発行メッセージ
- メッセージ通信順番は保証されない
 - ⇒FIFOトピックにより順番を保証可能に
- 取り消し不可
- 配信ポリシーによる再試行を実施
- メッセージサイズは最大256KB

FIFOトピック

FIFOトピックを利用することで配信順序通りに送信可能になる

SNSの標準トピック



FIFOトピック

FIFOトピックを利用することで配信順序通りに送信可能になる

SNSの標準トピック



SNSのFIFOトピック



SNSの特徴

SNSでは以下のようにHTTP/HTTPS/JSON形式のメッセージを利用している。

- HTTP/HTTPS ヘッダー
- HTTP/HTTPS 受信登録の確認の JSON 形式
- HTTP/HTTPS 通知の JSON 形式
- HTTP/HTTPS 受信登録の解除の JSON 形式
- SetSubscriptionAttributes 配信ポリシー JSON 形式
- SetTopicAttributes 配信ポリシー JSON 形式

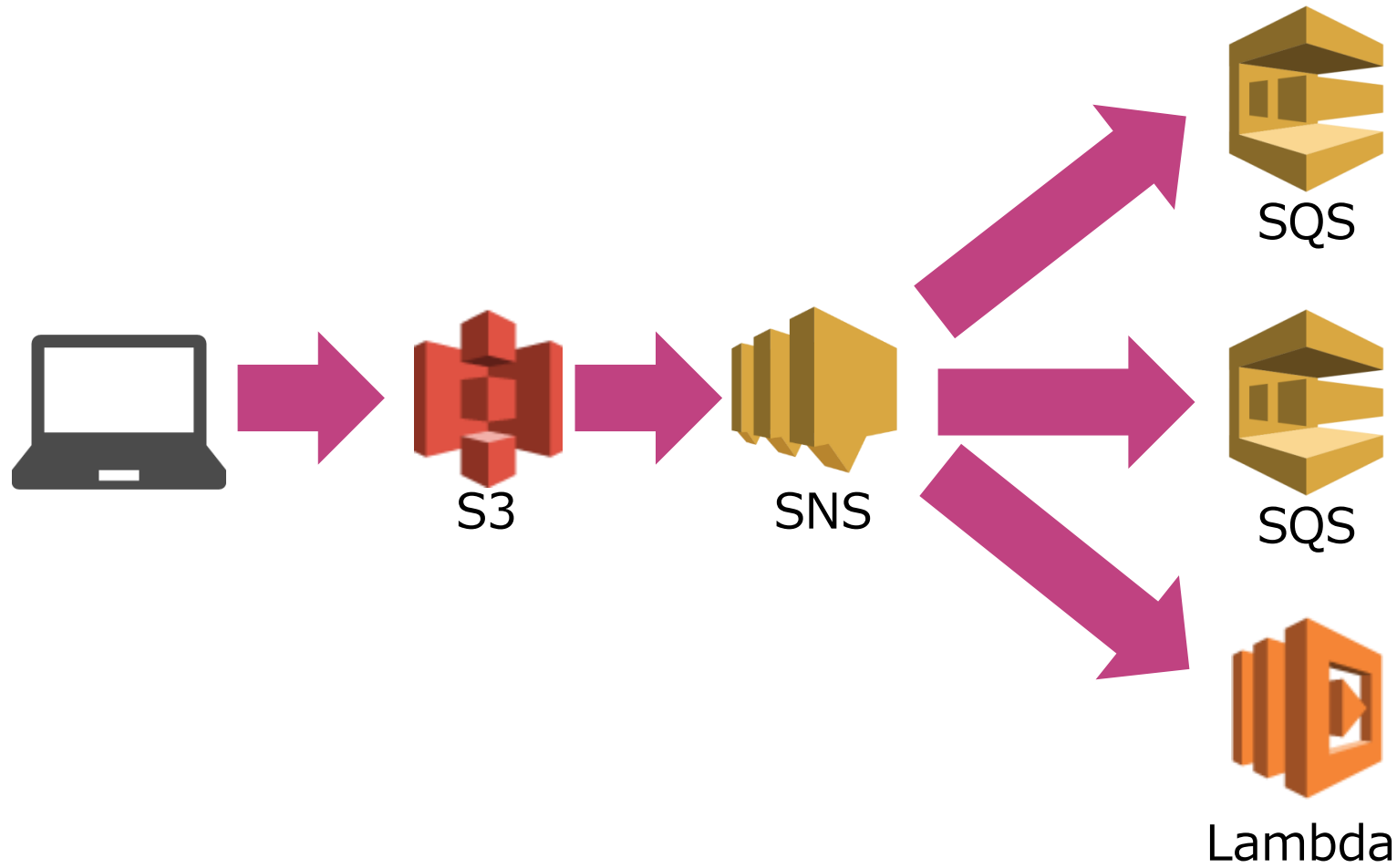
SNS連携

AWSの様々なサービスと連携して通知可能で、疎結合アーキテクチャに利用できる

- ❑ Amazon CloudWatch : Billing Alertの通知
- ❑ Amazon SES : Bounce/Complaintのフィードバック通知
- ❑ Amazon SQS : SNS通知によりキューを配信して、処理を実行
- ❑ Amazon S3 : ファイルがアップロードされた時の通知
- ❑ Amazon Elastic Transcoder : 動画変換処理完了/失敗時の通知
- ❑ AWS Lambda : SNSをトリガーとして処理を起動する。

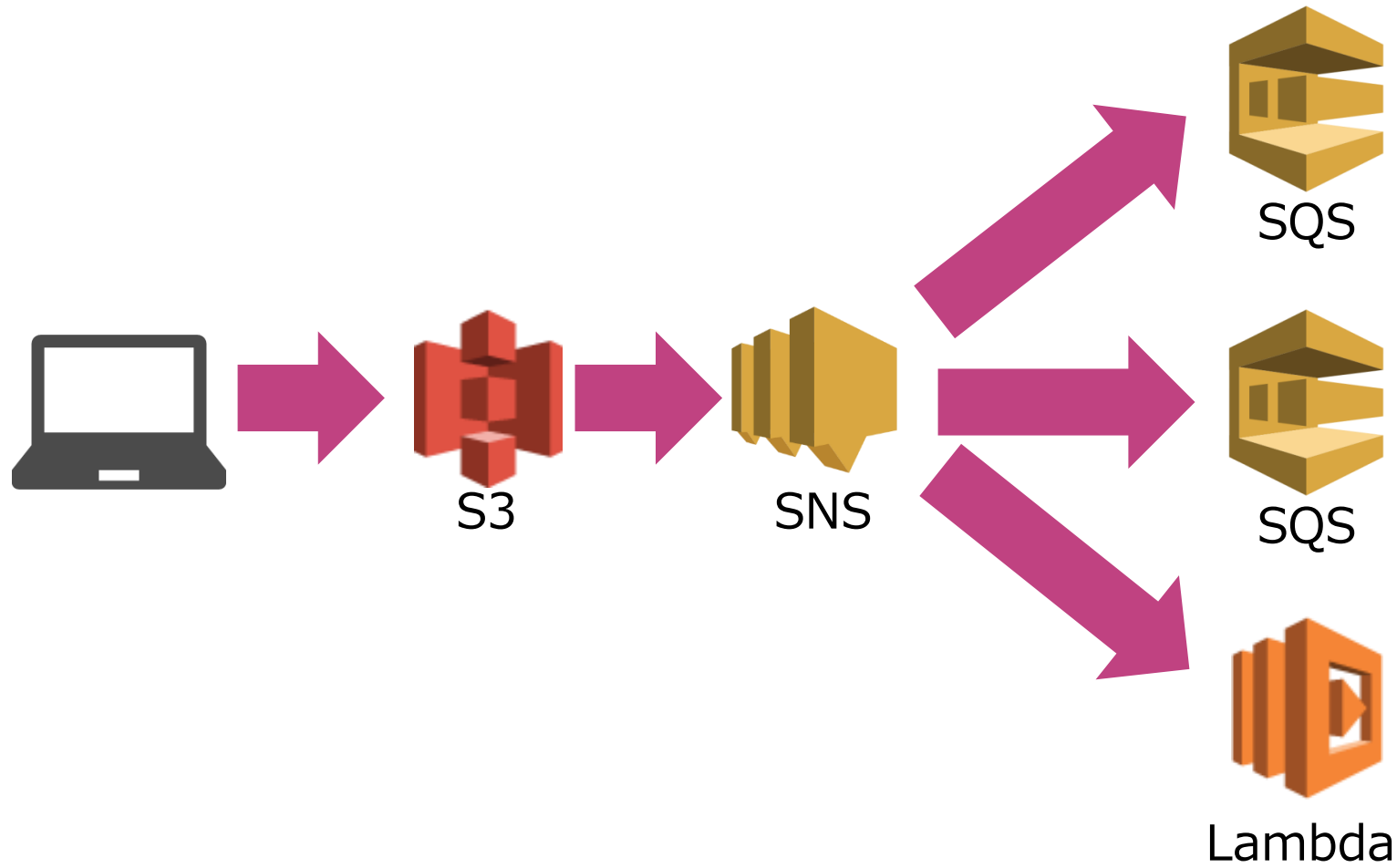
SNSとの連携

S3イベントと連携してSNSのプッシュ通知を配信することができる。



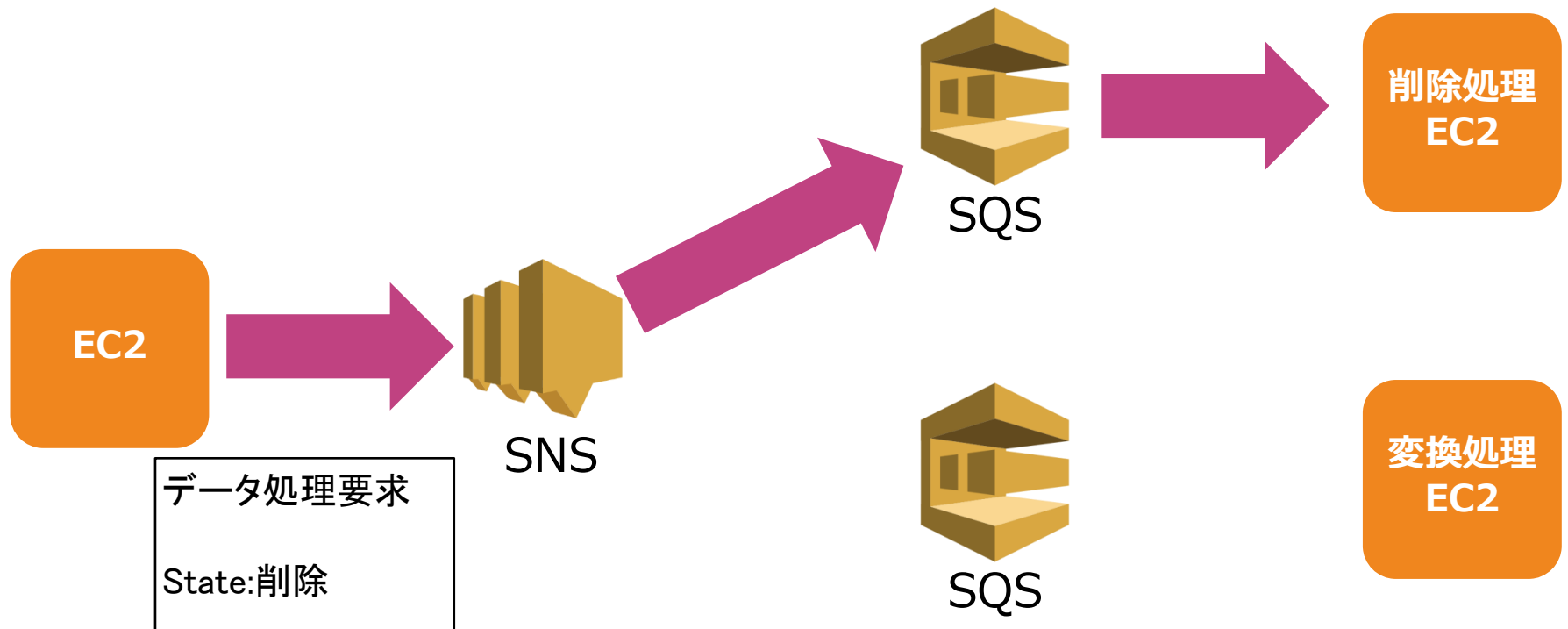
SNSとの連携

その中でSNSとSQSを掛け合わせた構成はファンアウト構成と呼ばれる



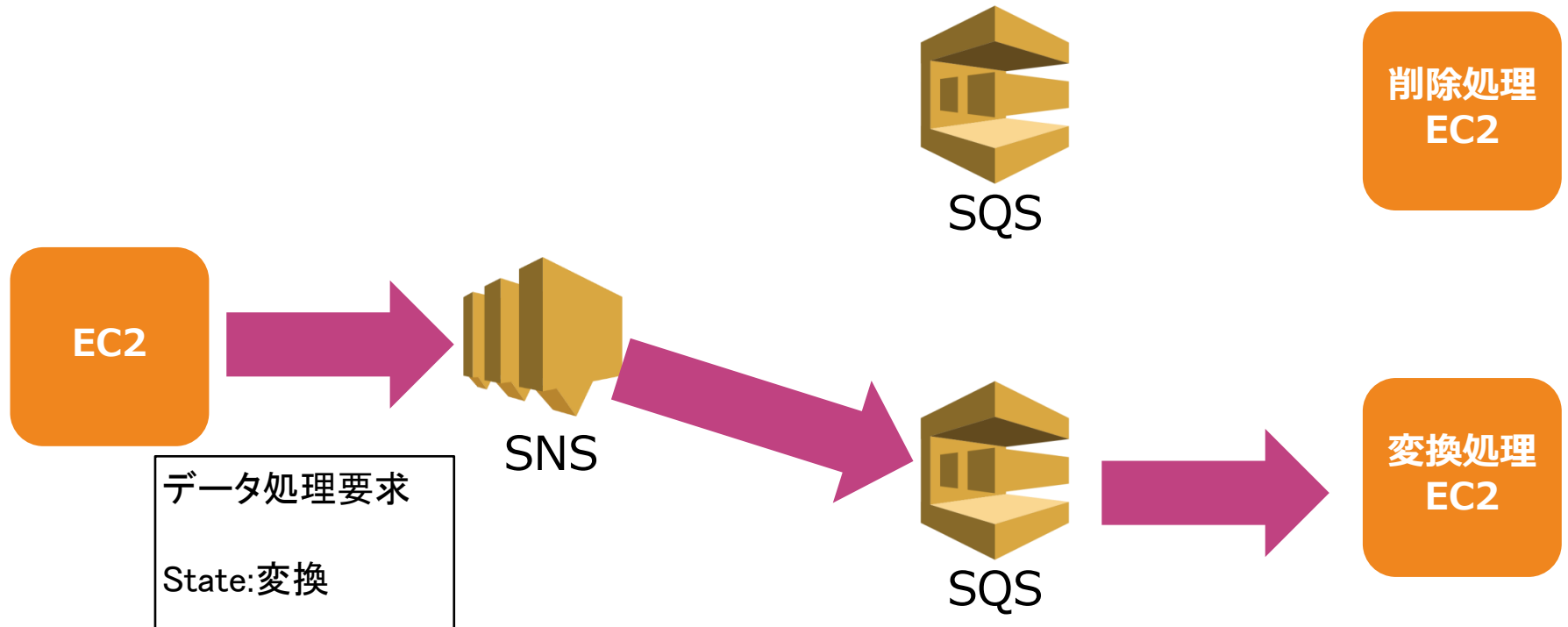
フィルタリング機能

フィルタリング機能を利用して送信先を特定して処理を限定することができる。



フィルタリング機能

フィルタリング機能を利用して送信先を特定して処理を限定することができる。



アクセス管理

SNSポリシーを利用して他アカウントや他リソースからのアクセスをコントロールすることができる。

IAMポリシー

- 一般的なIAMポリシーによるSNSリソースへのアクセス許可を設定する。
- SNSサービスへのアクセスを制御する。

SNSポリシー

- リソースベースポリシーとしてSNSリソース側でアクセス対象を制御するポリシー
- 他のアカウントとSNSを共有する許可が可能
- 他のサービスがSNSにメッセージを送信することを許可するなどの設定が可能

SESの概要

Amazon Simple Email Service(SES)

フルマネージド型／サーバレス型のコスト効率に優れたEメールサービス

- スケーラブルな構成で信頼性が高いマネージド型
- メール送信：トランザクションメールなどの高品質なコンテンツを顧客に送信可能
- メール受信：受信したメールをトリガーにS3やLambdaなどを起動可能
- バウンス処理：メールが送れなかった場合の処理を規定

SESのメール送信方法

単なるメールサーバーとして利用するだけでなく、アプリから自動でメール送信や連携処理に利用可能

HTTP REST API

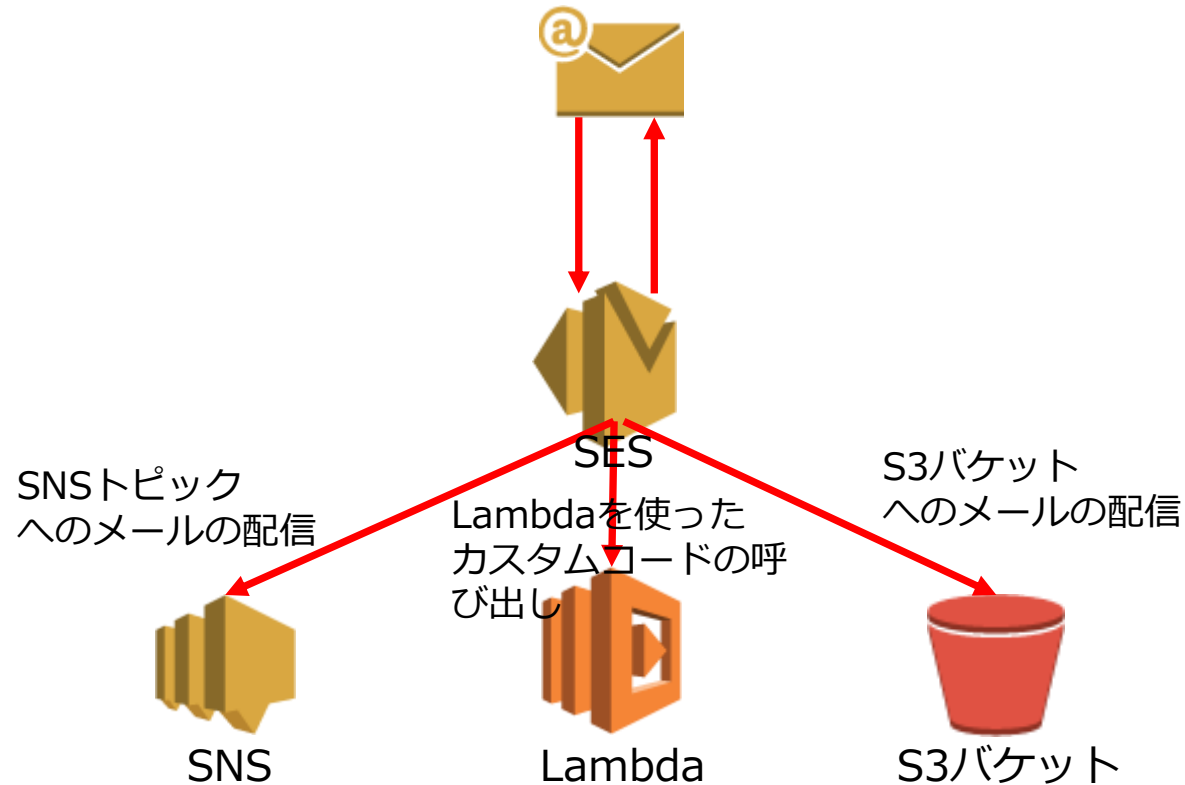
- ❑ SendEmail API : From/To/Subject/Bodyだけ用意すればSES側でメッセージを生成して送信する
- ❑ SendRawEmail API : メッセージ全体をアプリケーション側で生成して送信する
- ❑ 認証 : AWS アクセスキーとシークレットアクセスキーを使用

SMTPエンドポイント

- ❑ 生成済みEmailメッセージを受け取ってSESのSMTP エンドポイントを経由してメールを送信する。SMTPを前提としてプログラムから直接利用する場合などに利用
- ❑ 利用ポート
 - 25
 - 465(SMTP over SSL)
 - 587(Message Submission)
- ❑ TLS(Transport Layer Security) が必要
- ❑ 認証 : 専用IAMユーザを作成してそのクレデンシャルを使用

SESのメール受信

SESのメール通知を利用してS3やLambdaなどと連携が可能



SES利用準備

ドメイン登録やAWS上で利用できるメールとするための利用申請を事前にする必要がある

ドメインを登録

メールの事前申請

サーバレスによる
サービス化

サーバーではなくサービス（サーバレス）

マネージド型サービスとサーバレスアーキテクチャにより効率的な設計と運用を実現

主要なサーバレスサービス

Lambda

SNS

SQS

Dynamo
DB

SES

Step
Functions

Aurora
Serveless

API
Gateway

Amazon
Cognito

Amazon
S3

Fargate

Kinesis

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

アンマネージド型	
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定
運用 管理	ユーザーがソフトウェア など（インフラ以外）の 運用を実施
インフラ 管理	AWS側が実施する。

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

	アンマネージド型	マネージド型
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定
運用 管理	ユーザーがソフトウェア など（インフラ以外）の 運用を実施	AWS側が実施する。
インフラ 管理	AWS側が実施する。	AWS側が実施する。

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

	アンマネージド型	マネージド型	サーバレス
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	AWS側が実施する。
運用 管理	ユーザーがソフトウェア など（インフラ以外）の 運用を実施	AWS側が実施する。	AWS側が実施する。
インフラ 管理	AWS側が実施する。	AWS側が実施する。	AWS側が実施する。

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

	アンマネージド型 データベース	マネージド型 データベース	サーバレス データベース
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	AWS側が実施する。
運用 管理	ユーザーがソフトウェア など（EC2）の運用を 行う	AWS側が実施する。	AWS側が実施する。
インフラ 管理	AWS側が実施する。	AWS側が実施する。	AWS側が実施する。

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

	アンマネージド型 データベース	マネージド型 データベース	サーバレス データベース
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	AWS側が実施する。
運用 管理	ユーザーがソフトウェア など運用を 	AWS側が実施する。  Amazon Aurora	AWS側が実施する。
インフラ 管理	AWS側が実施する。	AWS側が実施する。	AWS側が実施する。

サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

	アンマネージド型 データベース	マネージド型 データベース	サーバレス データベース
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	AWS側が実施する。
運用 管理	ユーザーがソフトウェア など運用を 	AWS側が実施する。  Amazon Aurora	 Serverless
インフラ 管理	AWS側が実施する。	AWS側が実施する。	AWS側が実施する。

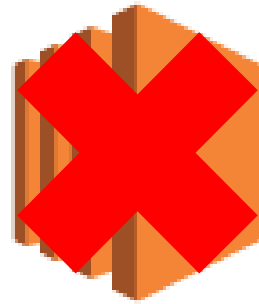
サーバレスとは何か？

サーバーを全く意識せずに利用できるPaaS型のクラウド機能

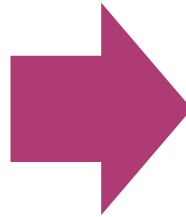
	アンマネージド型 コンピューティング	サーバレス コンピューティング
サーバー 設定	ユーザーがインスタンス タイプ選択などの仮想 サーバーを設定	AWS側が実施する。
運用 管理	ユーザーがソフトウェア など運用を 	AWS側が実施する。 
インフラ 管理	AWS側が実施する。	AWS側が実施する。

サーバレス化

サーバー（EC2インスタンス）ではなく、Lambdaなどのコンピュートサービスによるシステム構成を出来る限り利用する



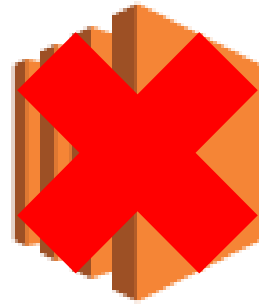
EC2
(サーバー)



Lambda
(コンピュートサービス)

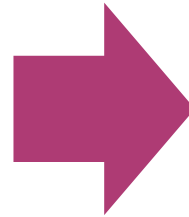
サーバレス化

サーバー（EC2インスタンス）ではなく、Lambdaなどのコンピュートサービスによるシステム構成を出来る限り利用する



EC2
(サーバー)

高コスト
密結合



Lambda
(コンピュートサービス)

低コスト
疎結合化

サーバレス化からサービス化へ

サーバレス設計によって疎結合設計が加速し、疎結合化によってマイクロサービス化されたアプリケーションが構築できる



サーバレス化

疎結合化

サービス化

サービス化

昔ながらのSOA方式から、より小さなプロセス単位のサービスをAPIで連携したマイクロサービスで設計する

サービス指向アーキテクチャ (SOA)

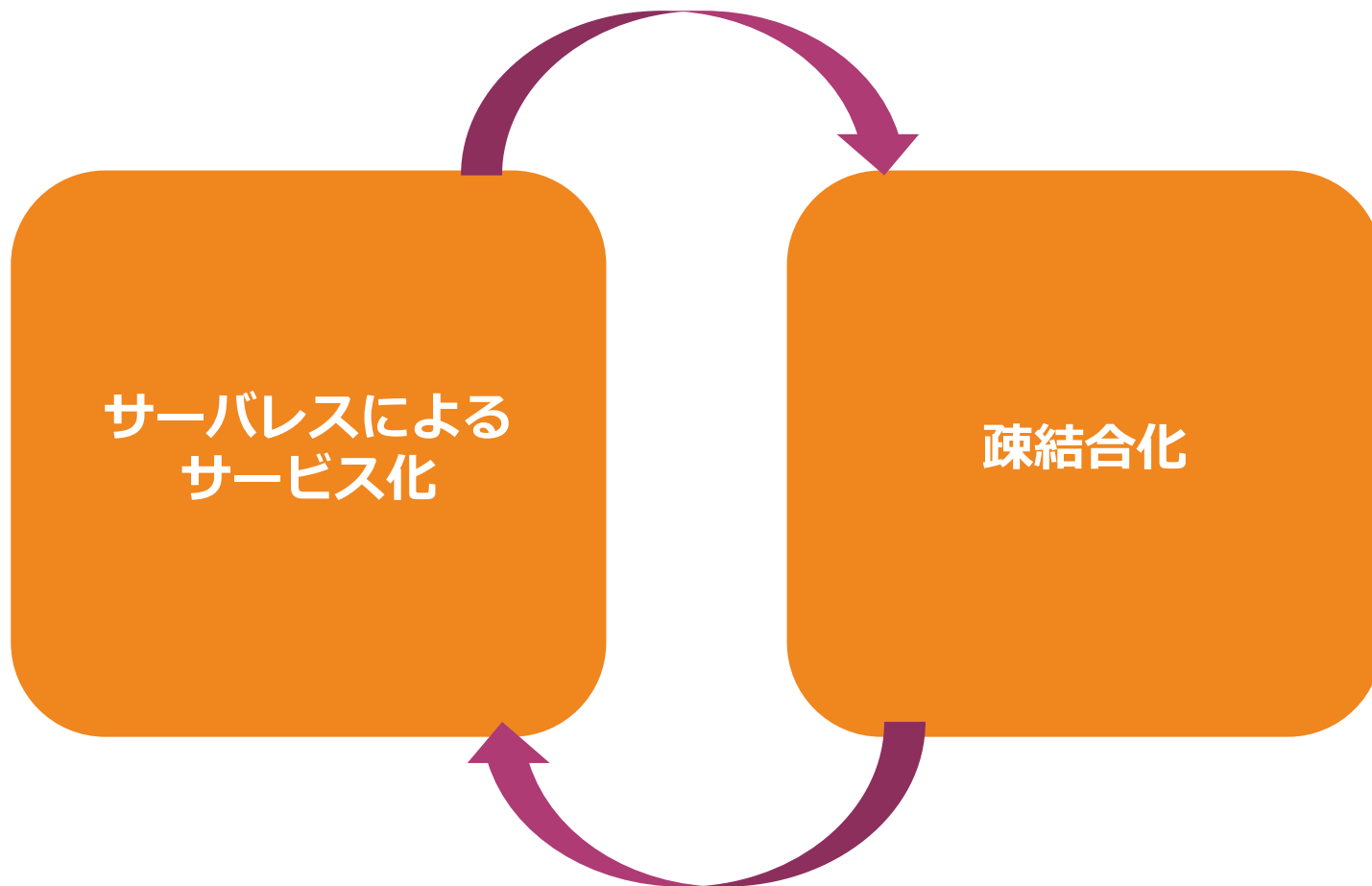
- ❑ 一通りの機能が揃った大きなサービス単位でコンポーネントを分割する設計方式
- ❑ アプリケーションをコンポーネント化して、通信プロトコルによる連携で他のコンポーネントにサービスを提供するアーキテクチャ設計
- ❑ ex.年金システムであれば給付サービス・徴収サービス・適用サービスなど

マイクロサービス

- ❑ SOAよりも小さな機能単位のプロセスでサービス化した構成
- ❑ 各プロセスでは1つの小さなタスクをサービスとして提供し、プロセス間通信はAPIを用いる

サービス化

疎結合化とマイクロサービス化は表裏一体の関係となる



サーバレス化のポイント

利用しているインスタンスが本当に必要か設計を見直す

シンプルな機能や処理を実行するだけなのにインスタンスを利用している・・・など

Lambdaの概要

Lambdaとは何か？

サーバーを起動せずにプログラミングコードを実行する仕組み。
簡易なアプリケーション処理を構築することができる。



Lambdaとは何か？

サーバーを起動せずにプログラミングコードを実行する仕組み。
簡易なアプリケーション処理を構築することができる。

DynamoDBへの許可
ポリシーが設定された
IAMロールを付与

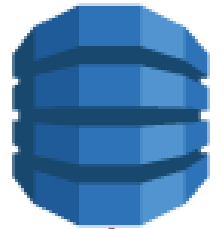


Lambdaがファンクション（関数）を実行

Lambdaファンクション

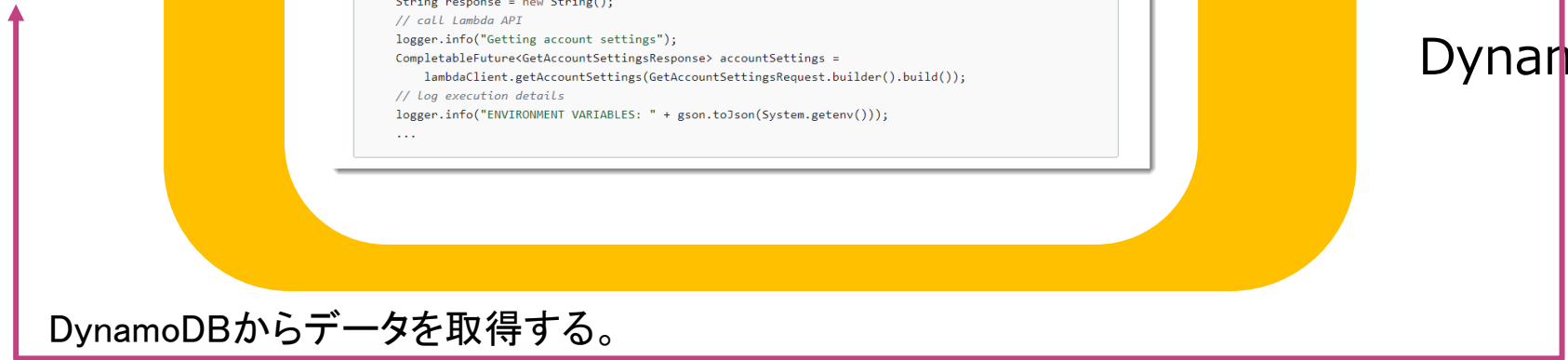
Example [Handler.java](#) – Initialization Code

```
// Handler value: example.Handler
public class Handler implements RequestHandler<SQSEvent, String>{
    private static final Logger logger = LoggerFactory.getLogger(Handler.class);
    private static final Gson gson = new GsonBuilder().setPrettyPrinting().create();
    private static final LambdaAsyncClient lambdaClient = LambdaAsyncClient.create();
    ...
    @Override
    public String handleRequest(SQSEvent event, Context context)
    {
        String response = new String();
        // call Lambda API
        logger.info("Getting account settings");
        CompletableFuture<GetAccountSettingsResponse> accountSettings =
            lambdaClient.getAccountSettings(GetAccountSettingsRequest.builder().build());
        // Log execution details
        logger.info("ENVIRONMENT VARIABLES: " + gson.toJson(System.getenv()));
        ...
    }
}
```



DynamoDB

DynamoDBからデータを取得する。



Lambdaの特徴

Lambda関数は様々なコードを利用可能で、AWS側で実行環境が管理されている。

- ❑ 実行基盤は全てAWS側で管理されているマネージド型サービス
- ❑ AWSサービスと連携させることでLambda関数（ファンクション）と呼ばれる簡単にイベントドリブンなアプリケーションを実装可能
- ❑ **Java、Go、PowerShell、Node.js、C#、Python、Ruby** のランタイムをサポート
【関数の内容】
- ❑ コード
関数のコードと依存関係を作る。スクリプト言語の場合は、組み込みエディタで関数コードを編集が可能。ライブラリを追加するには、またはエディタでサポートされていない言語の場合は、デプロイパッケージをアップロードする。デプロイパッケージのサイズが 50 MB を超える場合はS3からアップロードする。
- ❑ ランタイム – 関数を実行する Lambda ランタイムのこと
- ❑ ハンドラー – 関数の呼び出し時にランタイムで実行されるメソッド

Lambdaの課金

Lambdaはリクエスト数とコードの実行期間で算出されて課金される。

- ❑ コード実行時間に対しての課金されるため、サーバーを保持して処理コードを実行するよりもコスト効率が非常に高い。
- ❑ リクエストの数とコードの実行時間に基づいて課金
- ❑ 実行時間はコードの実行が開始された瞬間から処理が返されるか、中止されるまでの時間で計算される。値は100 ミリ秒単位で切り上げられます。
- ❑ 1 か月ごとに 100 万件の無料リクエスト、および 40 万 GB-秒のコンピューティング時間が無料枠になっている。

Lambdaの制限

Lambda関数は効率的な処理を可能にするために、データ量や実行時間や同時実行数に制限がある。

- ❑ 関数のタイムアウト時間は**デフォルト値は3秒で、許容されている最大値は900秒**（15分）。タイムアウトに達すると、関数が停止される。
- ❑ 関数の最大同時実行数はデフォルトは100を最大は1000（申請によって数十万まで引き上げ可能）
- ❑ 関数の実行時に使用できるメモリの量。メモリの量を 128 MB ～ 10,240MB の範囲
- ❑ /tmp ディレクトリのストレージの保存可能容量は 512 MB～10,240 MB
- ❑ Lambdaレイヤーを最大5つまで設定可能

Lambdaの仕組み

ユーザー側はコードをLambda環境で実装するだけでアプリケーションの機能を開発できる。

```
graph TD; A[Lambdaファンクションを用意する  
(コーディング)] --> B[Lambda環境でコードを呼び出す]; B --- C[AWSが管理するLambda関数を実行する  
サーバー環境];
```

Lambdaファンクションを用意する
(コーディング)

Lambda環境でコードを呼び出す

AWSが管理するLambda関数を実行する
サーバー環境

Lambdaの仕組み

ユーザー側はコードをLambda環境で実装するだけでアプリケーションの機能を開発できる。

ユーザー側はサーバーを意識しない

Lambdaファンクションを用意する
(コーディング)

Lambda環境でコードを呼び出す

AWSが管理するLambda関数を実行する
サーバー環境

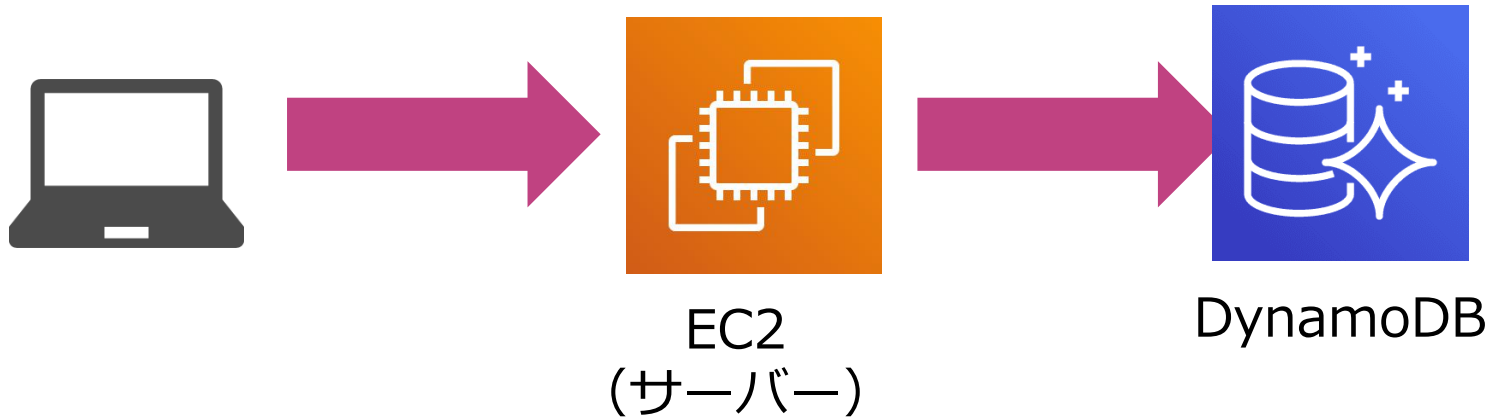
Lambdaとの連携

Lambda関数は様々なAWSサービスをトリガーに起動したり、連携することができる。

- ❑ Amazon S3
- ❑ Amazon Kinesis
- ❑ Amazon DynamoDB Streams
- ❑ Amazon Cognito(Sync)
- ❑ Amazon SNS
- ❑ Amazon SQS
- ❑ Alexa Skills Kit
- ❑ Amazon SWF

Lambdaユースケース

クライアントからのアクセスに対してデータ登録する単純な処理をEC2インスタンスに実行させているならば・・・



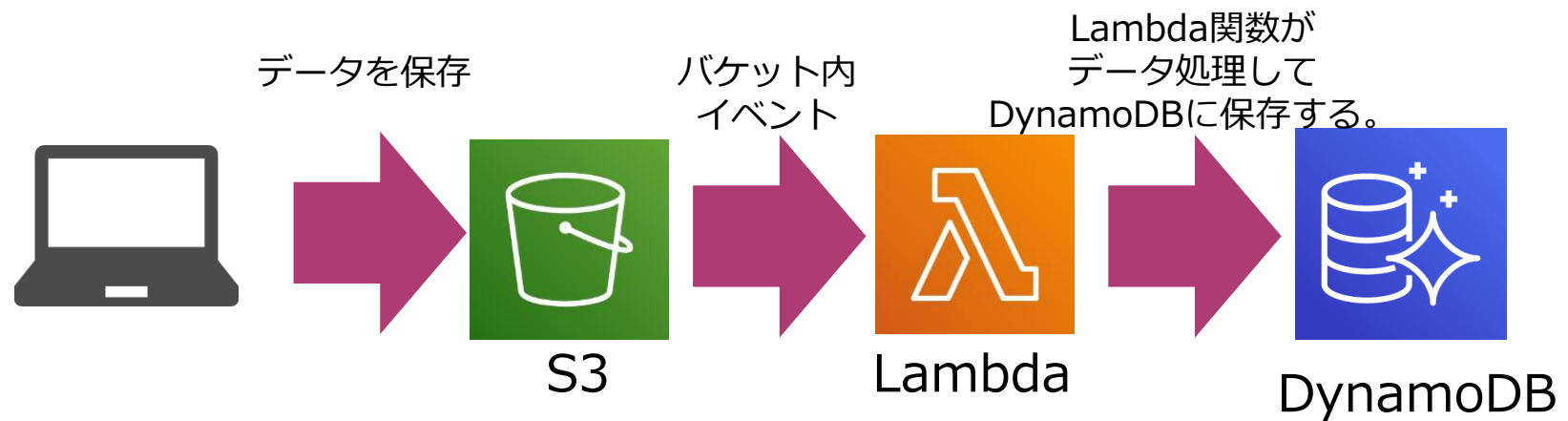
Lambdaユースケース

Lambdaに置き換えてサーバレスに実行処理することが可能



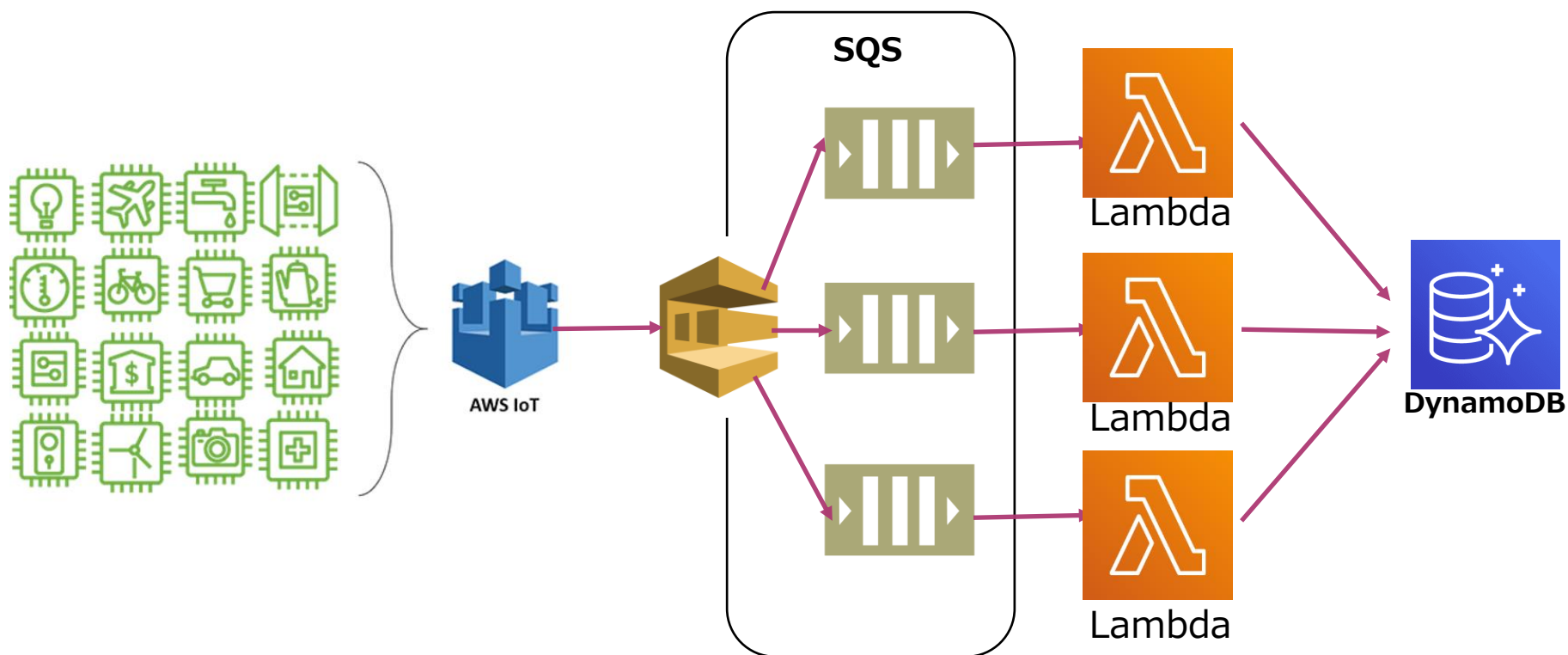
Lambdaユースケース

S3イベントと連携して、Lambdaを起動してデータを処理する。



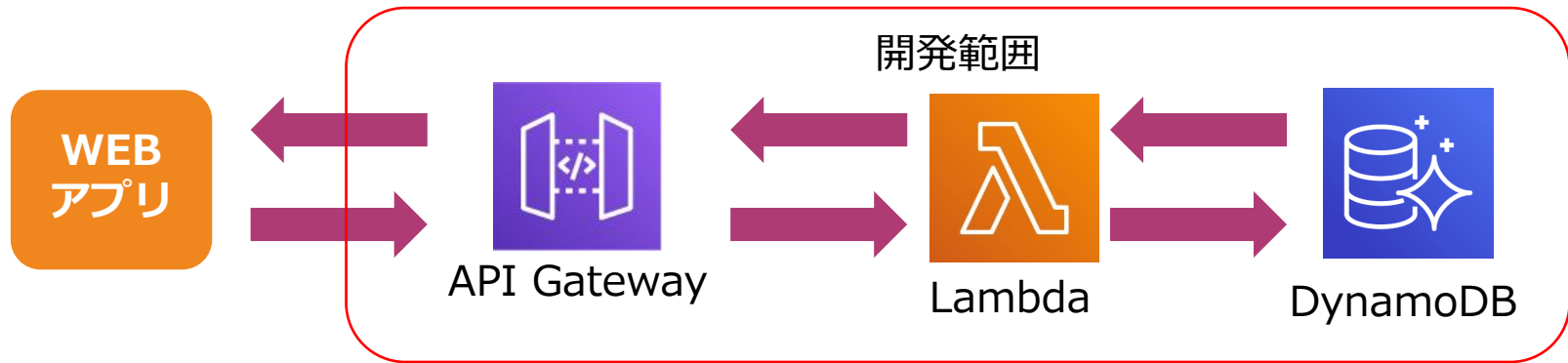
Lambdaユースケース

SQSとLambdaを組み合わせることでIoTセンサーデータを
DynamoDBに格納する処理プログラムを作ることが可能



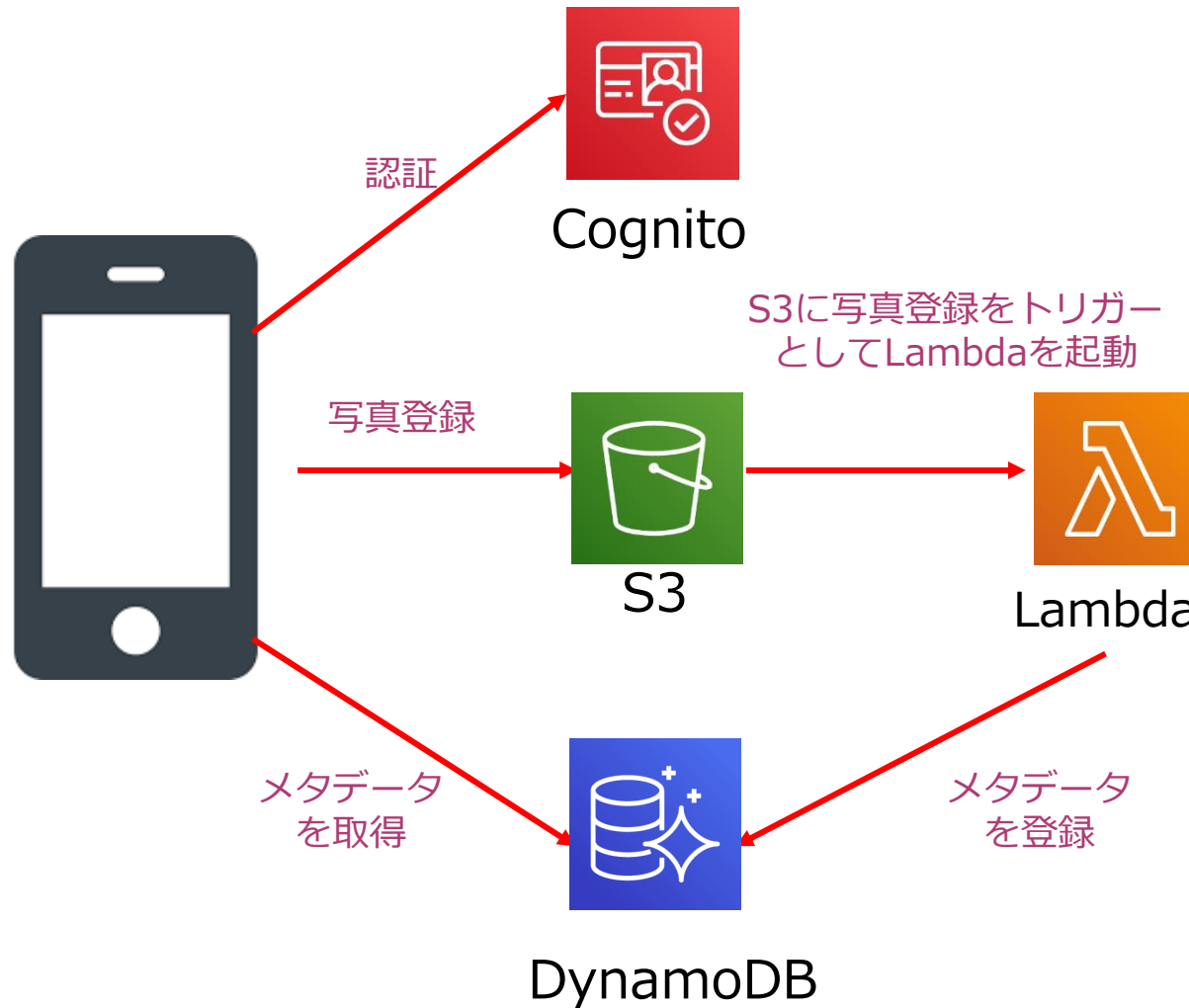
API Gatewayとの連携

API Gatewayと統合することで、Lambda関数をAPIから実行することができる。



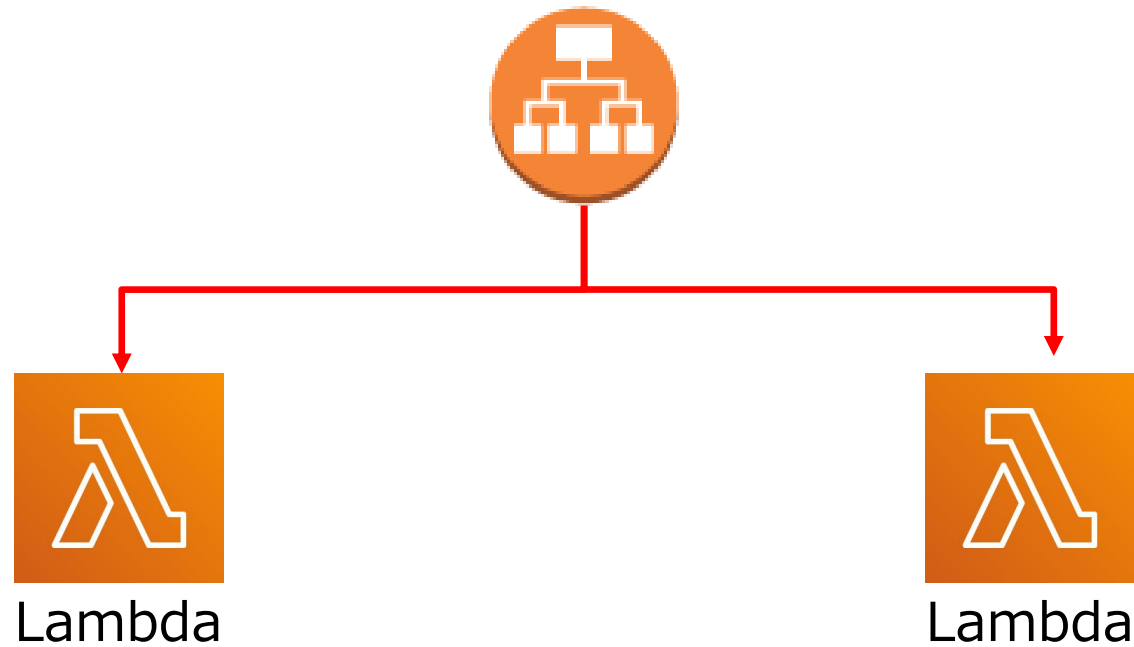
Lambdaモバイルアプリ

モバイルアプリの写真管理をLambdaにより実施するなどのモバイル連携も容易



ロードバランサー機能

ALBのバックエンドにLambda関数を構成することが可能になり、WEBアプリにLambda関数を組込易くなった。



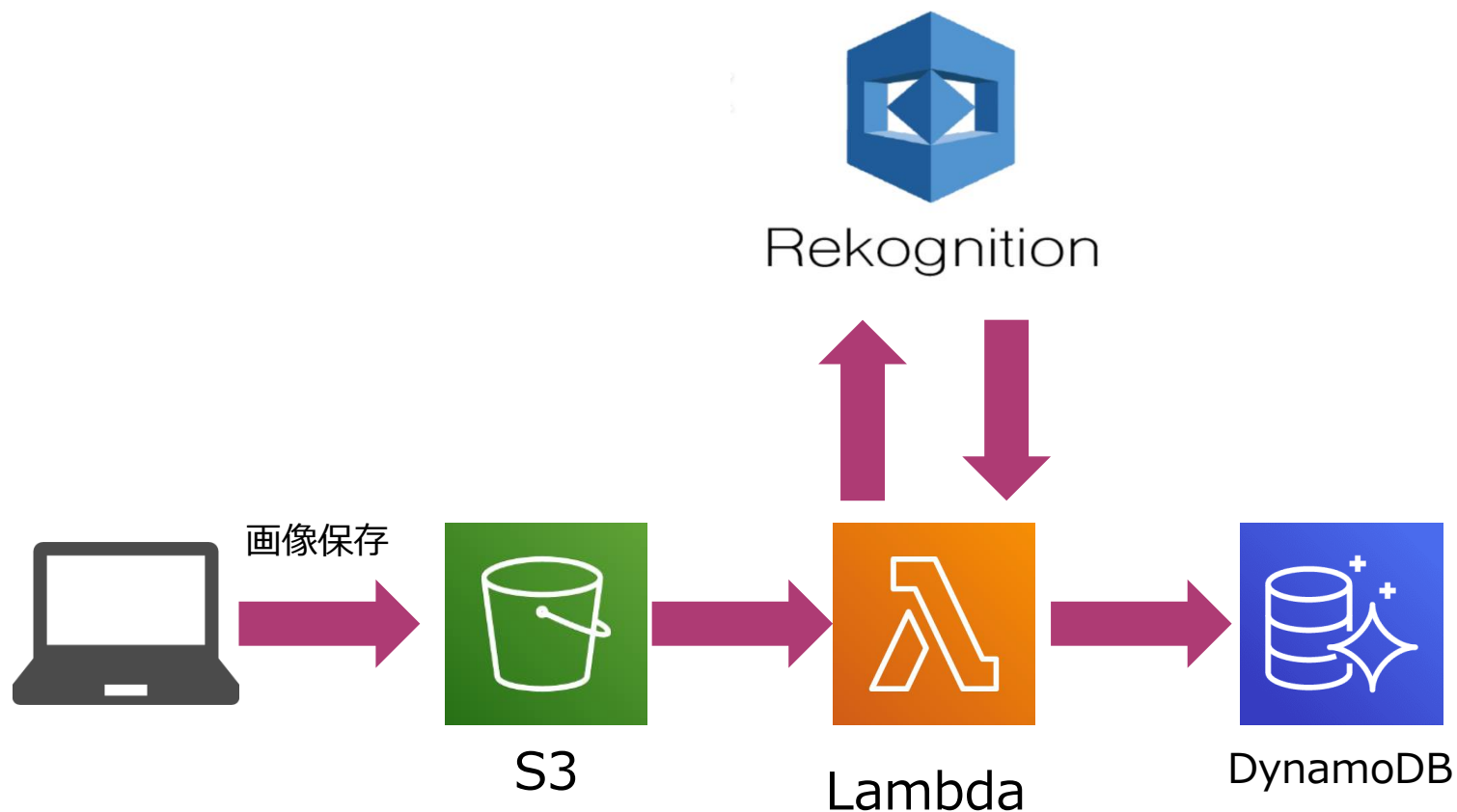
Lambdaユースケース

Amazon Echoの音声処理をトリガーとしてAlexaスキルを呼び出すことができる。



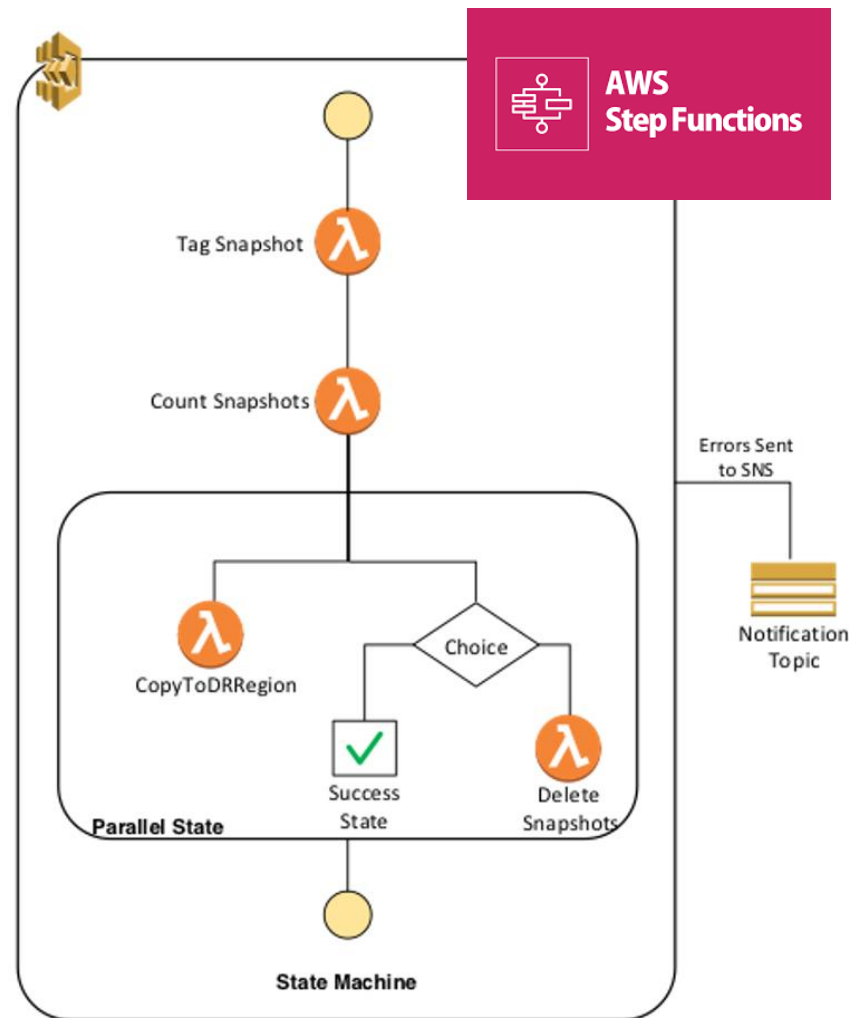
Lambdaユースケース

Lambda関数からRekognitionなどのAIサービスのAPIを呼び出して、AI機能をサーバレスで実行する。



Lambdaユースケース

AWS StepFunctionsを利用して複数のLambda関数を組み合わせたワークフローを構築できる。



Lambdaの実装：ブループリント

Lambdaファンクションをコーディングする際にサンプルコード集を利用することが可能



Lambdaを利用
するユースケース
を設計

ブループリントに
てサンプルコード
を探す

サンプルコードを
修正してファンク
ションを作成する

AWS Lambda アプリケーション

Lambda 関数をイベントソースやその他のリソースを組み合わせたもので、協調して動作することによりタスクを実行する。

AWS Serverless Application Repository

- ❑ 数クリックでアカウントにデプロイできる Lambda アプリケーションのコレクションを提供
- ❑ リポジトリにはプロジェクトの開始点として使用できるアプリケーションとサンプルが保存されている。

AWS SAM + CloudFormation

- ❑ AWS SAMを利用してLambdaアプリケーションを展開する。
- ❑ AWS SAMはLambdaはAWS CloudFormation を拡張して、Lambdaアプリケーションを展開する。

Lambdaの処理タイミング

他のAWSサービスやSDKを利用したアプリケーションからの呼び出して実行することが可能

非同期呼び出し

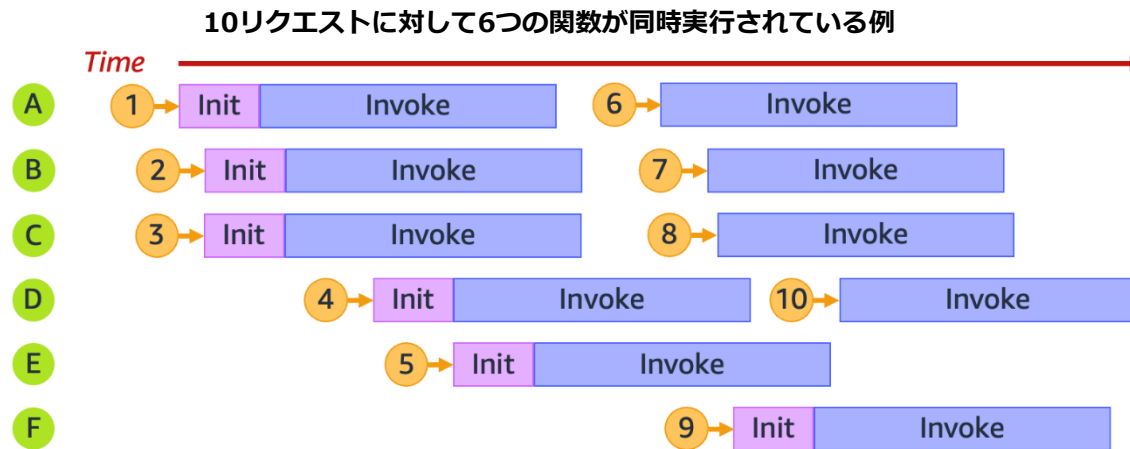
- ❑ 関数を非同期的に呼び出してイベントを処理する。
- ❑ 関数を非同期的に呼び出す場合は、関数コードからのレスポンスを待機しない。

同期呼び出し

- ❑ 関数を同期的に呼び出すと、Lambdaが関数を実行し、レスポンスを待つ。
- ❑ 実行完了時に、実行された関数のバージョンなどの追加データとともに、Lambda関数内でセットしたレスポンスが返ってくる

Lambdaのスケーリング

Lambda関数はオートスケールとキャパシティ予約が可能。



- Lambda は、同時実行リクエストごとに、実行環境の個別のインスタンスをプロビジョニングして自動でスケールする（デフォルトで1,000まで）
- 予約同時実行では、関数の同時インスタンスの最大数を予め予約しておいて、保証する。メリットは、他の関数が関数のスケールを妨げないことと、関数のスケールが制御不能にならないこと。
- プロビジョニング済み同時実行では、リクエストされた数の実行環境を初期化して、関数の呼び出しに即座に応答する準備しておく

Lambdaレイヤー

Lambdaファンクション間で共通するコンポーネントを
Lambda Layerとして定義し参照できる（5つまで）

Lambda
ファンク
ション

共通機能

Lambda
ファンク
ション

共通機能

Lambda
ファンク
ション

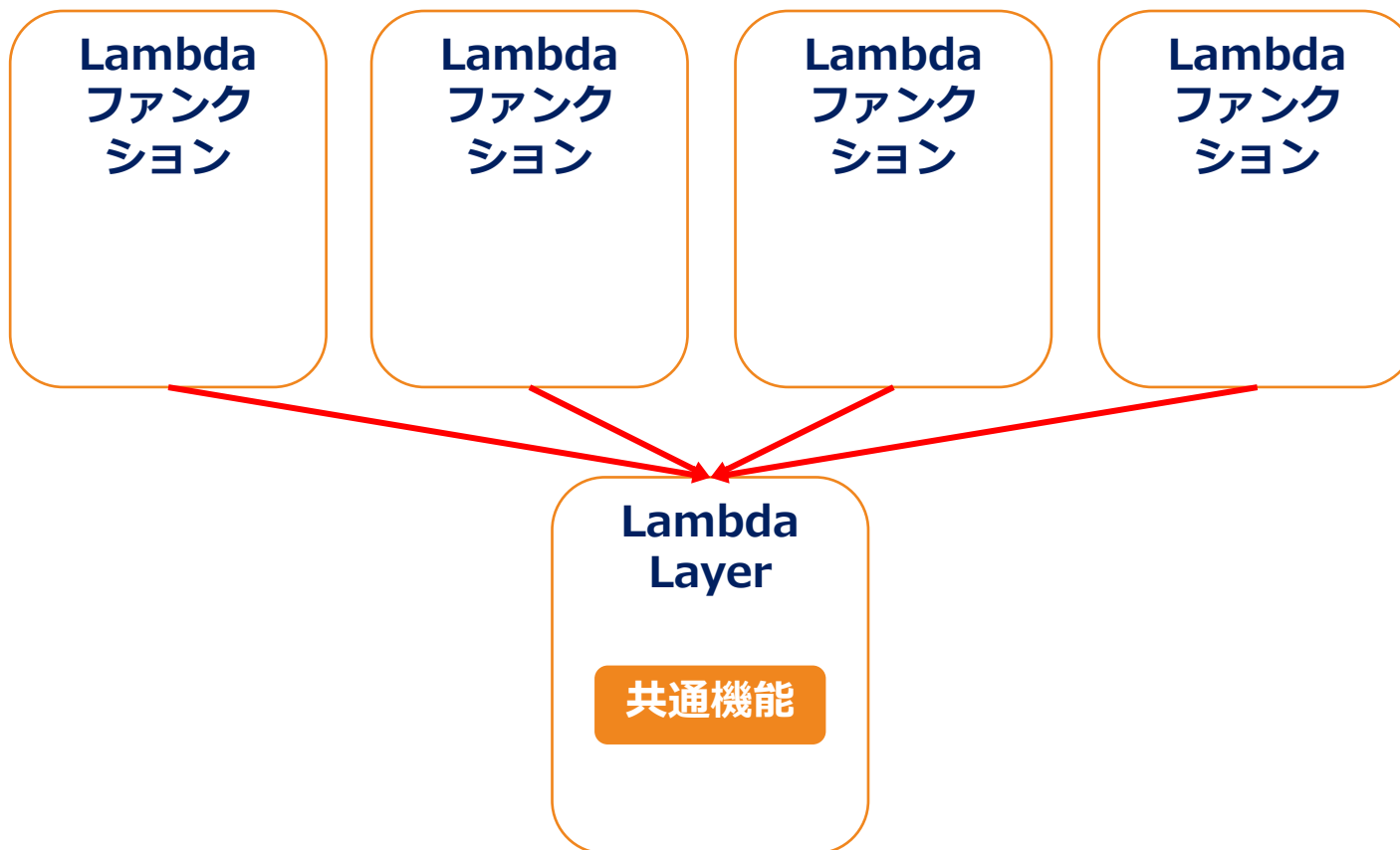
共通機能

Lambda
ファンク
ション

共通機能

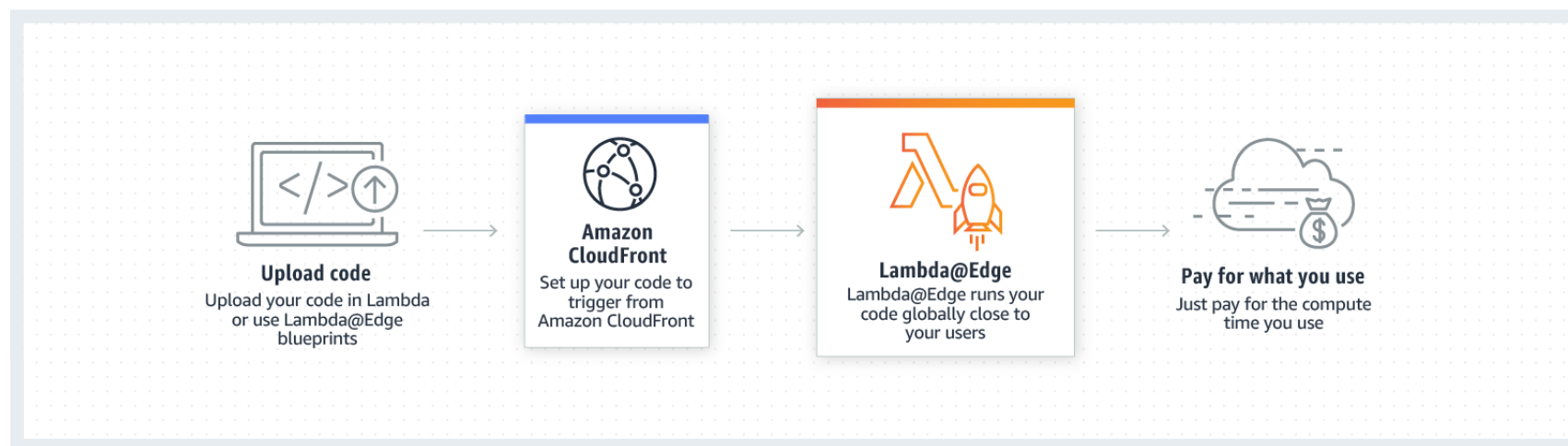
Lambdaレイヤー

Lambdaファンクション間で共通するコンポーネントを
Lambda Layerとして定義し参照できる（5つまで）



Lambdaエッジ

CloudFrontの配信コンテンツをLambda関数によってエッジロケーションで処理することが可能

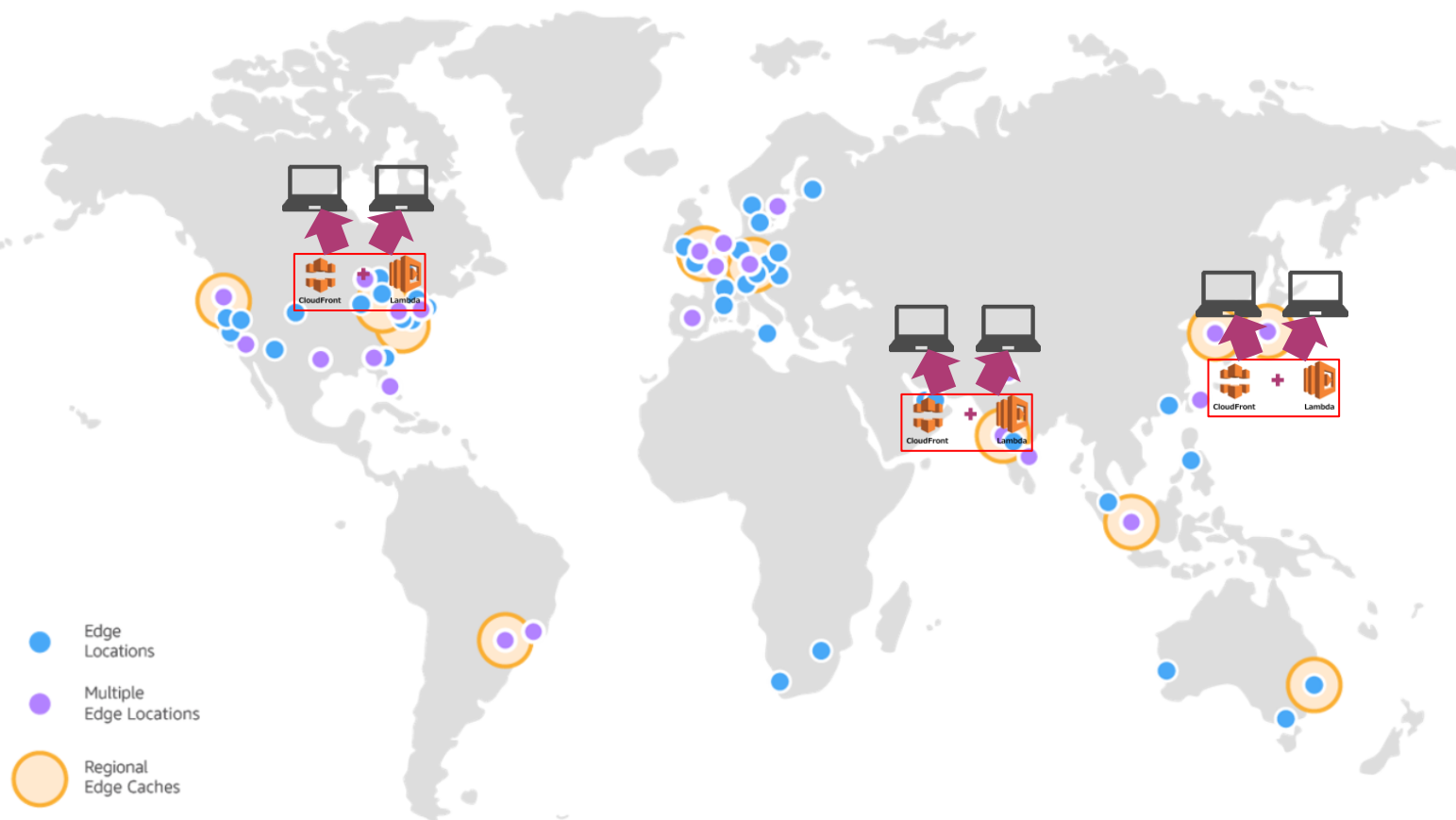


Reference: <https://aws.amazon.com/jp/lambda/edge/>

- グローバルにLambda関数を並行実施したい場合
- ユーザーに近いロケーションでコンテンツ配信などのフィルタリング処理などを実施する場合

Lambdaエッジ

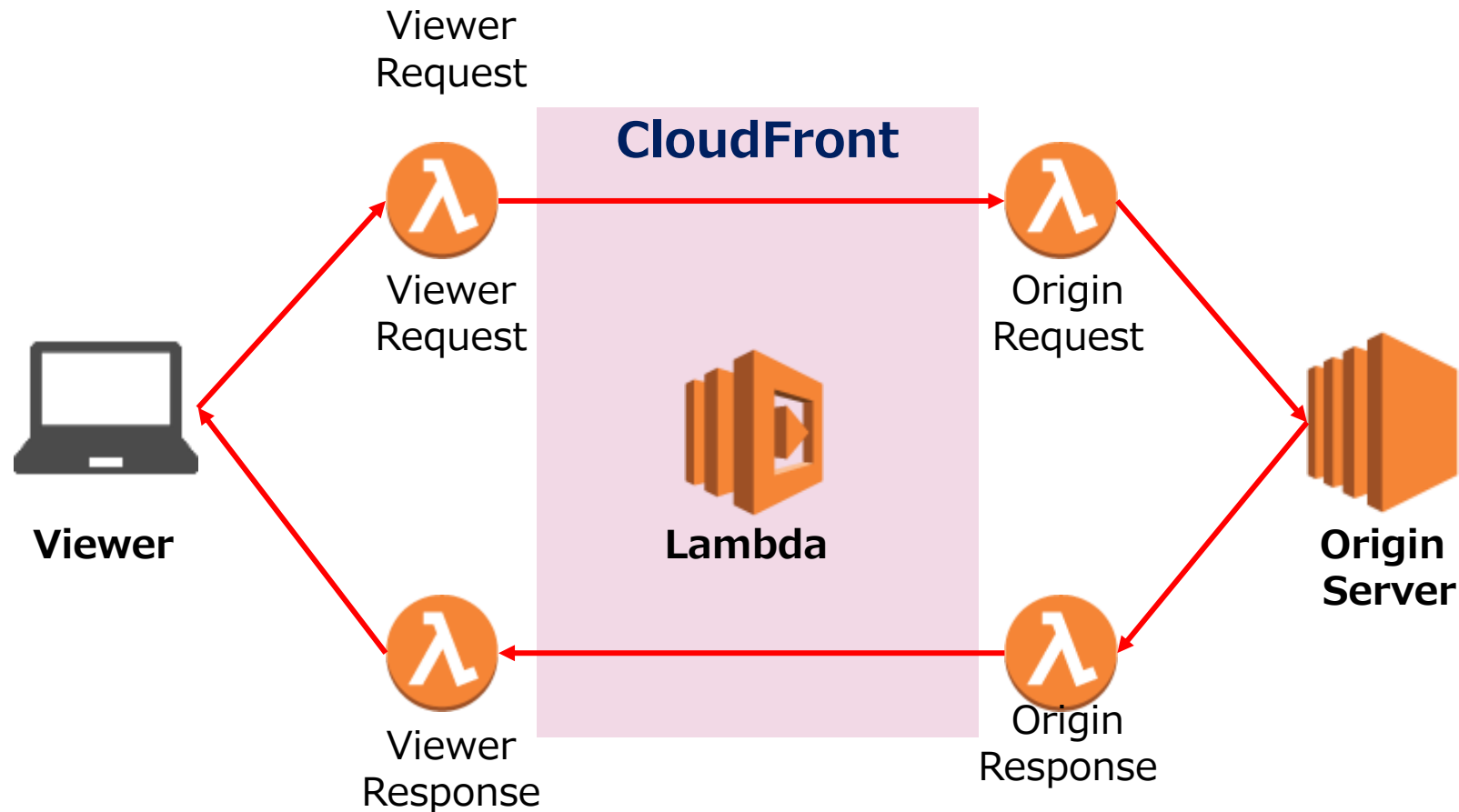
CloudFrontにLambda機能を連携することで、世界中でユーザーに近いロケーションにおいてコードを実行できる



参照: <https://aws.amazon.com/jp/cloudfront/features/?nc=sn&loc=2>

Lambdaエッジ

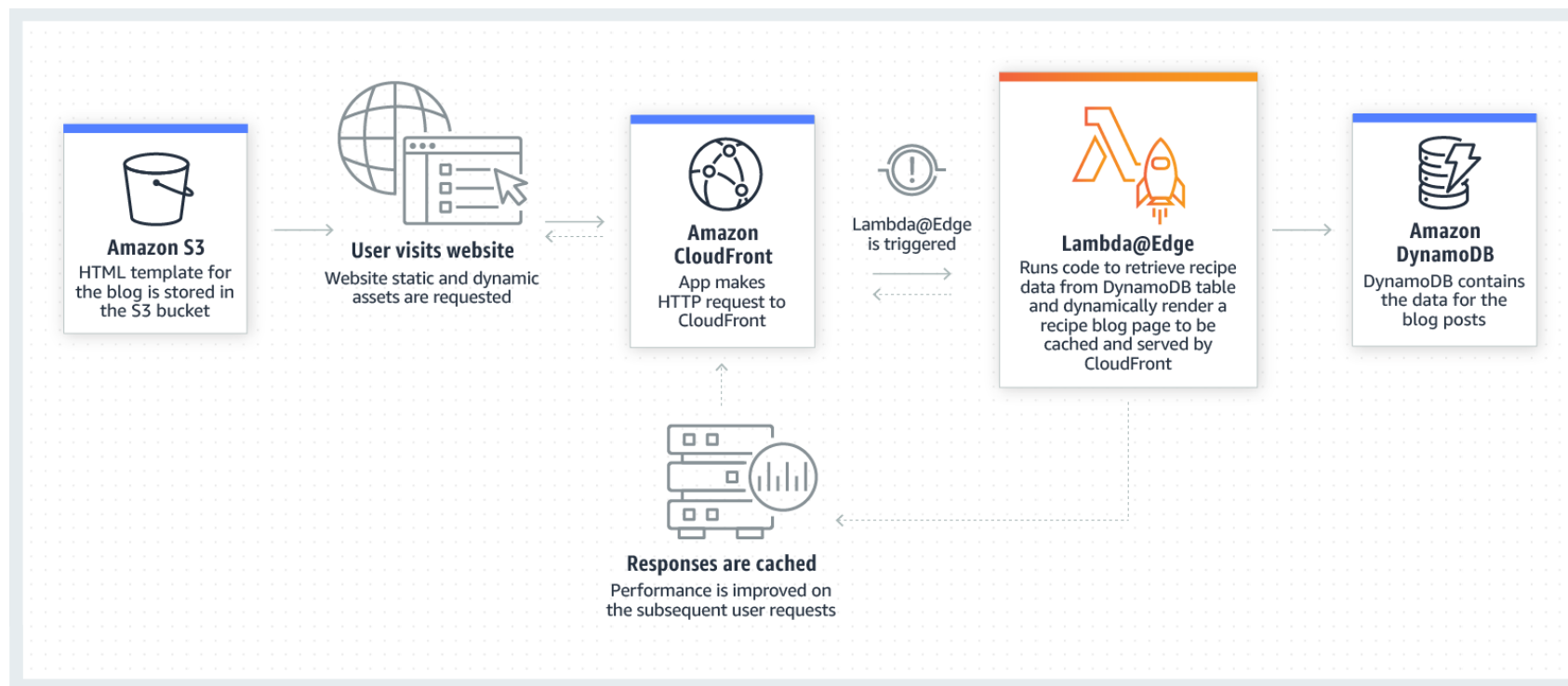
イベントに関連付けられてLambdaファンクションがエッジロケーションで実行されて実行結果を返答する



- CloudFrontのレスポンス/リクエスト処理時にフィルタリングなどの関数処理を実行することができる。

Lambda@Edgeのユースケース

オリジン上のアプリケーションコードを変更することなく、すべてのオリジン応答に HTTP セキュリティヘッダーを追加して、セキュリティとプライバシーを向上させる



参照 : <https://aws.amazon.com/jp/lambda/edge/>

VPCアクセス

インターネットを経由せずにVPC内のAWSリソースへとアクセス可能になる

VPC内のリソース へのアクセス

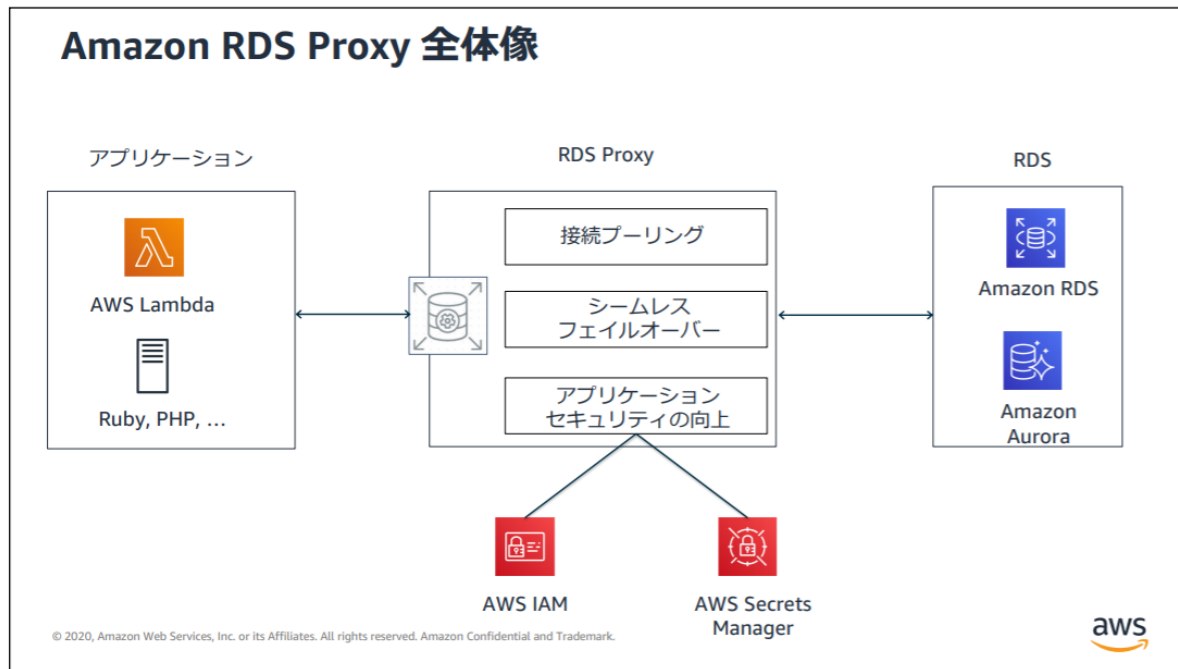
- ❑ VPC内リソースにインターネットを経由せずにアクセスが可能
- ❑ VPCを指定する際にサブネットIDとVPCセキュリティグループIDを指定して、ENIを作成する。ENI経由で接続
- ❑ ENIには指定したサブネットのIPがDHCPで動的に割り当てられる

アクセス設定

- ❑ ファンクションに割り当てるIAM Roleに“AWSLambdaVPCAccessExecutionRole”というポリシーをアタッチしておくこと

RDSプロキシ

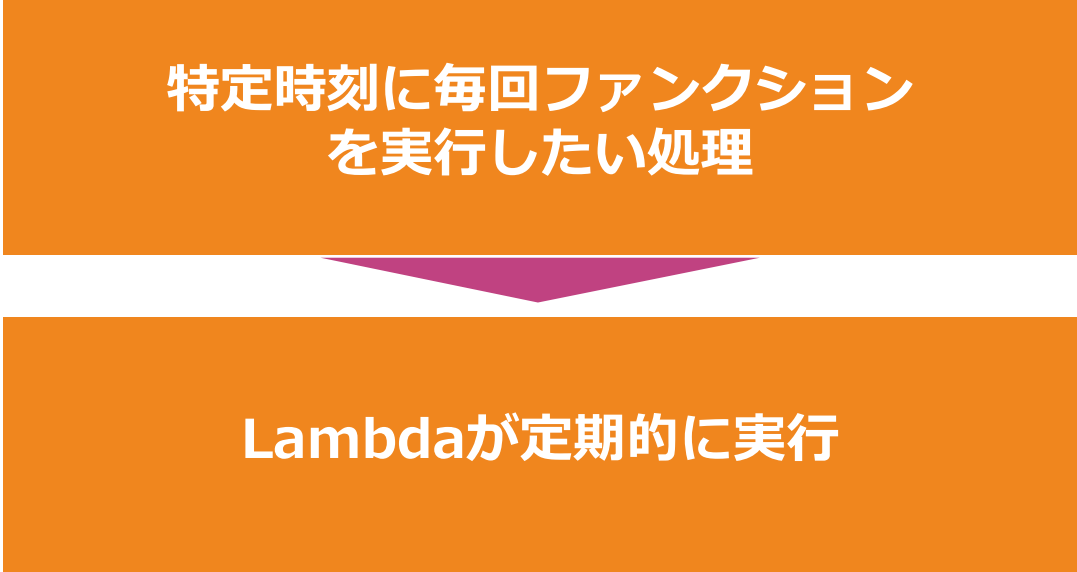
Lambdaを利用してRDSのデータベースに接続する際は、RDSプロキシをエンドポイントの代わりに利用して、接続をプーリングすることで効率化できる。



- プロキシを利用して接続をプーリングして無駄なコネクションを削減
- RDS Proxy への接続はIAM認証を利用する
- TLS/SSLによる暗号化を実施
- フェールオーバーによる高可用な接続をマネージド型で提供

スケジュール機能

特定時刻をトリガーにしてLambdaファンクションを実行する



```
graph TD; A[特定時刻に毎回ファンクション  
を実行したい処理] --> B[Lambdaが定期的に実行]
```

特定時刻に毎回ファンクション
を実行したい処理

Lambdaが定期的に実行

バージョンニング

Lambda関数のバージョン管理を実施して、特定時点のバージョンを複数保持・管理できる。

バージョンの発行方法

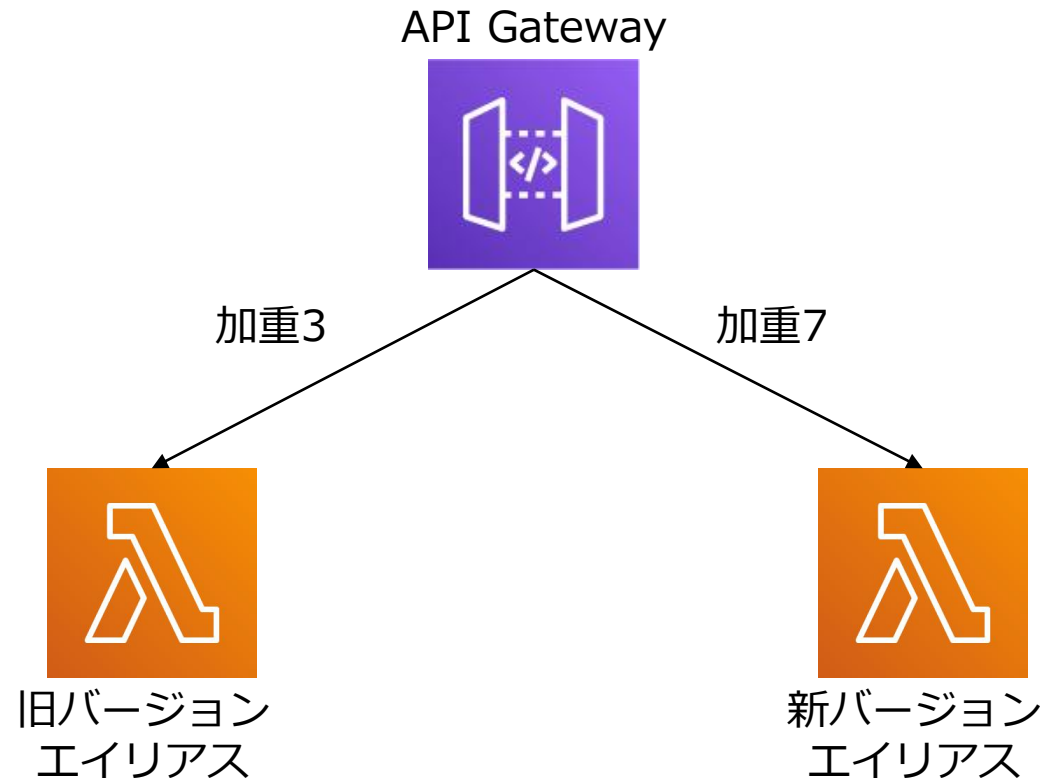
- ❑ Lambda関クションの作成や更新時にpublishパラメータによりバージョンが発行される
- ❑ PublishVersionにより明示的にバージョンを発行することが可能

バージョンの特徴

- ❑ 一度発行すると変更不可
- ❑ 単純にバージョン番号が増加する
- ❑ エイリアス（特定バージョンに対するポインタ）を設定して特定時点にマークすることが可能
- ❑ エイリアスを作成することでバージョン番号を把握していなくても指定バージョンを呼び出せる

バージョンニング

エイリアス毎に加重を設定してバージョン更新を調整する



コード署名

Lambda関数にコード署名を付与することで、コードが変更されず、信頼された発行元から提供されていることを証明する。

▼ 詳細設定

☒ コード署名を有効化 [Info](#)

コード署名設定を使用して、コードが承認されたソースによって確実に署名されており、かつ、署名後に変更されていることのないようにします。

コード署名設定

署名検証ポリシーとコードの署名を許可されている署名プロファイルを定義する設定を選択します。

新しい設定を作成



説明

このコード署名設定をリストで表示するか、関数に追加するときに、このコード署名設定を識別する説明を入力します。

最大長は 256 文字です。

署名プロファイル [Info](#)

署名プロファイルのバージョン ARN を選択または入力します。署名プロファイルを作成するには、[AWS Signer](#) を使用します。

署名プロファイルバージョン ARN

`arn:aws:signer:region:1234567890:/signing-profiles/example/abcdef`



署名プロファイルの追加

署名プロファイルを 19 件追加できます。

コードの署名検証ポリシー [Info](#)

☒ Warn

デプロイは成功し、CloudWatch に警告が記録されます。

☐ Enforce

未承認のプロファイルによってコードが署名された場合、または署名の有効期限が切れたり取り消されたりした場合、デプロイをブロックします。

Code.zip ロケーション

☒ Amazon S3

☐ ローカル

AWS Signer

AWS Signerはコードの信頼性と整合性を確保するためのフルマネージコード署名サービス

AWS Lambdaの コード署名

- ❑ コード署名により信頼できるコードのみがLambda関数で実行されるようにする。
- ❑ AWS Signerを使用して、Lambdaデプロイ用のデジタル署名されたパッケージを作成できる。

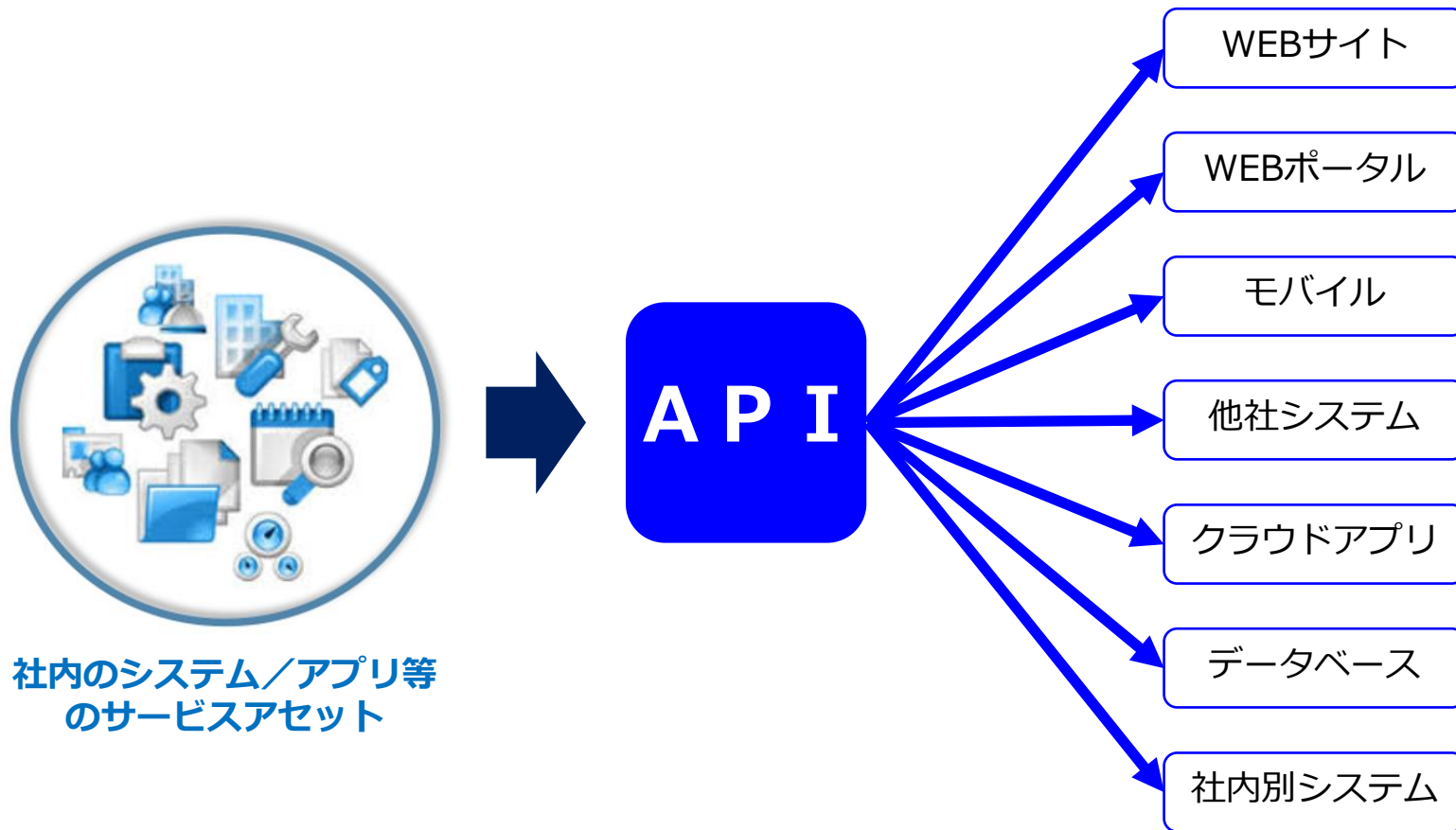
AWS IoTの コード署名

- ❑ AWS IoTのコード署名を使用すると、AmazonFreeRTOSおよびAWSIoTデバイス管理でサポートされているIoTデバイス用に作成したコードに署名して信頼性を担保する。
- ❑ AWS Certificate Manager (ACM) と統合されており、ACMを使用してサードパーティのコード署名証明書をインポートする。

APIとは何か？

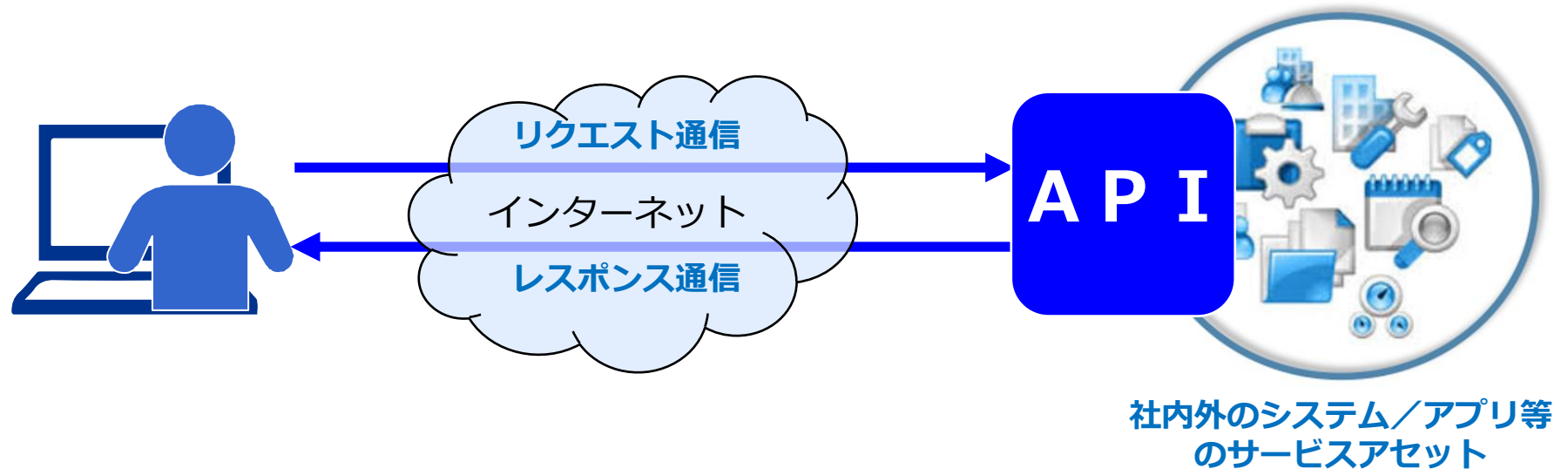
APIとは

API (Application programming interface) はシステムとシステムをつなぐ連結器



APIとは

APIを通してリクエストとレスポンスにより、他サービスの機能やデータを呼び出すことができる



【専門的には】

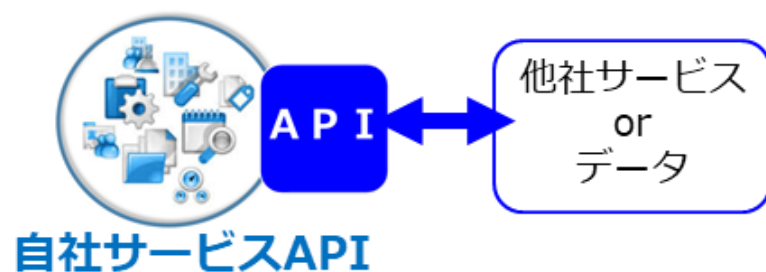
- ✓ APIはアプリケーションソフトウェア開発に利用される標準的なインターフェース群のこと
- ✓ 中でもWEB APIはWEB上で他のサービスを呼び出す方式や取り決め

APIの活用

自社アプリ・サービスをAPI化して連携を目指す方法と、API化された他社アプリ・サービスを活用する方法がある

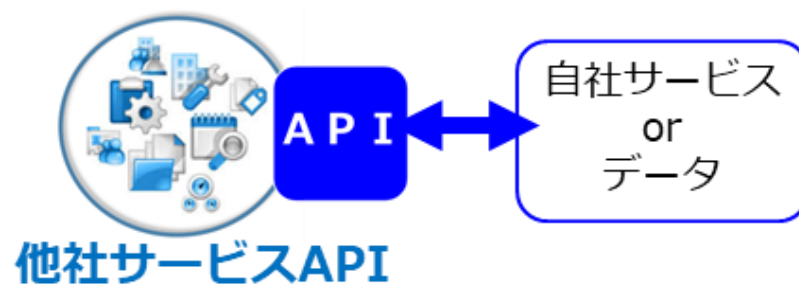
自社アプリ・サービスのAPI公開

自社アプリ・サービスをAPI化して、他社サービスとの連携してサービスを展開する



他社アプリ・サービスのAPI活用

他社APIサービスを活用して、自社事業と組合せたアプリやサービスを展開する



APIエコノミー

APIエコノミーと自社サービスやデータをAPI化して社内外連携を促進し、ビジネス領域・価値を拡大させるビジネス活動

他社サービスと連携した経済圏（エコノミー）の拡大

A社モバイルアプリ

C社アプリ



API

自社アプリAPI

B社ECサービス

D社サービス

API利用の必要事項

APIの活用のためにはAPIを効率的に構築・運用・保守が十分にできる必要がある

APIの作成

API利用状況の監視

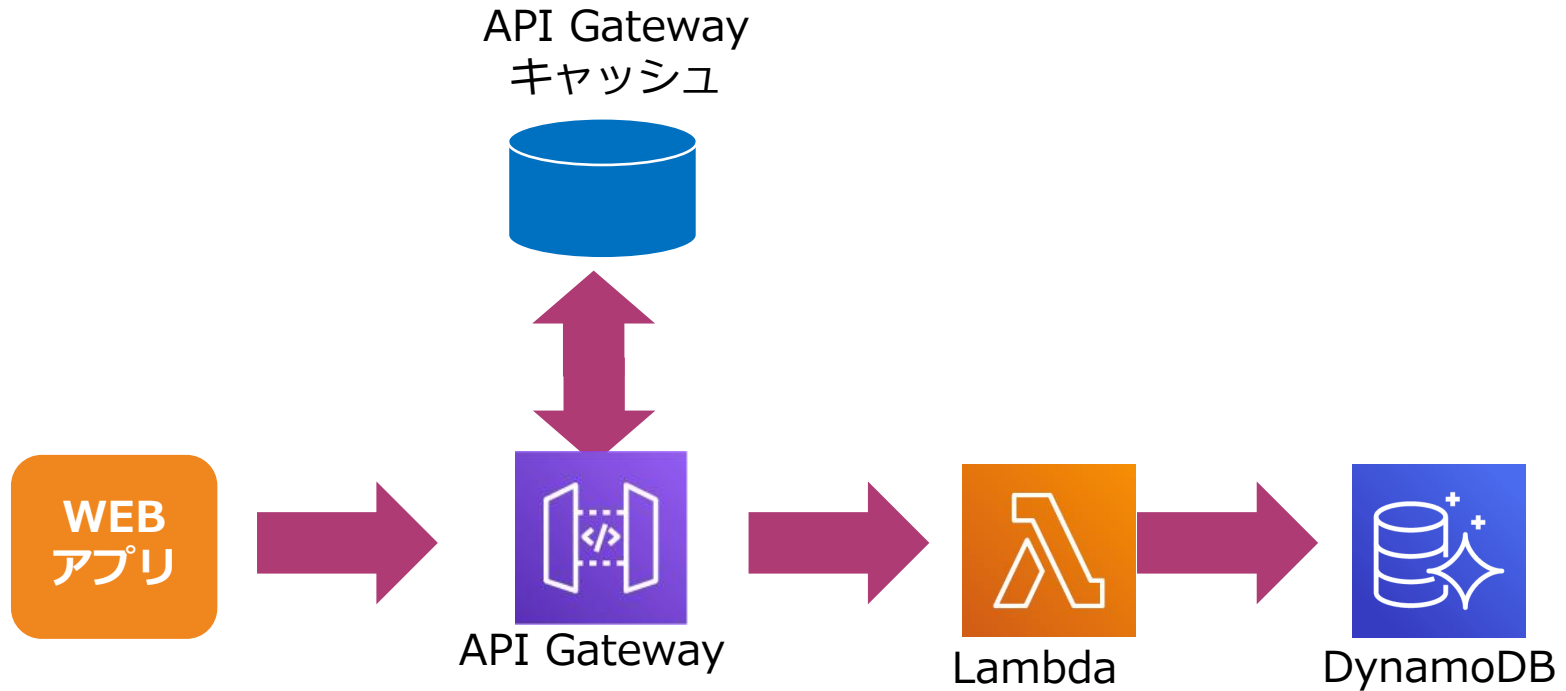
APIのバージョン管理

APIの認証・アクセス管理

API Gatewayの概要

API Gateway

API GatewayからLambda関数を起動してサーバレスアプリケーションを実行する。



API利用の必要事項

APIの活用のためにはAPIを効率的に構築・運用・保守が十分にできる必要がある

APIの作成

API利用状況の監視

APIのバージョン管理

APIの認証・アクセス管理

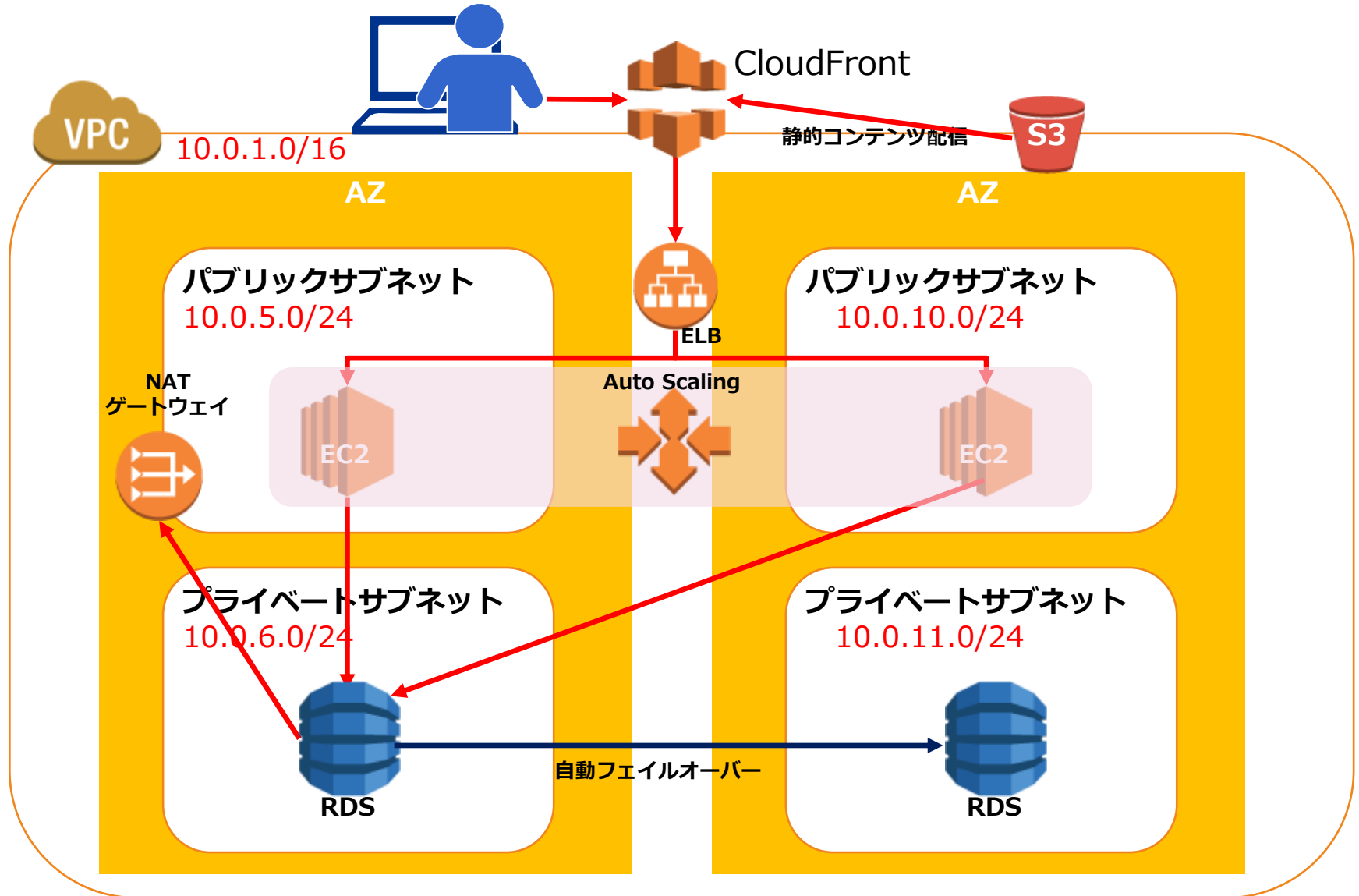
API Gateway

API GatewayはAPI作成/管理をフルマネージド型サービスで提供

- 最大数十万個のAPI同時呼び出し・受付が可能
- アクセス制御の管理
- DDoS攻撃対応やスロットリングによるバックエンド保護
- EC2/Lambda/任意のウェブアプリケーションのワークロード処理を実行する
- Lambdaと密接に統合されている
- Swagger/Open APIのインポートによってAPIを定義
- WebSocketを利用したリアルタイムかつ双方向通信のAPIも処理可能

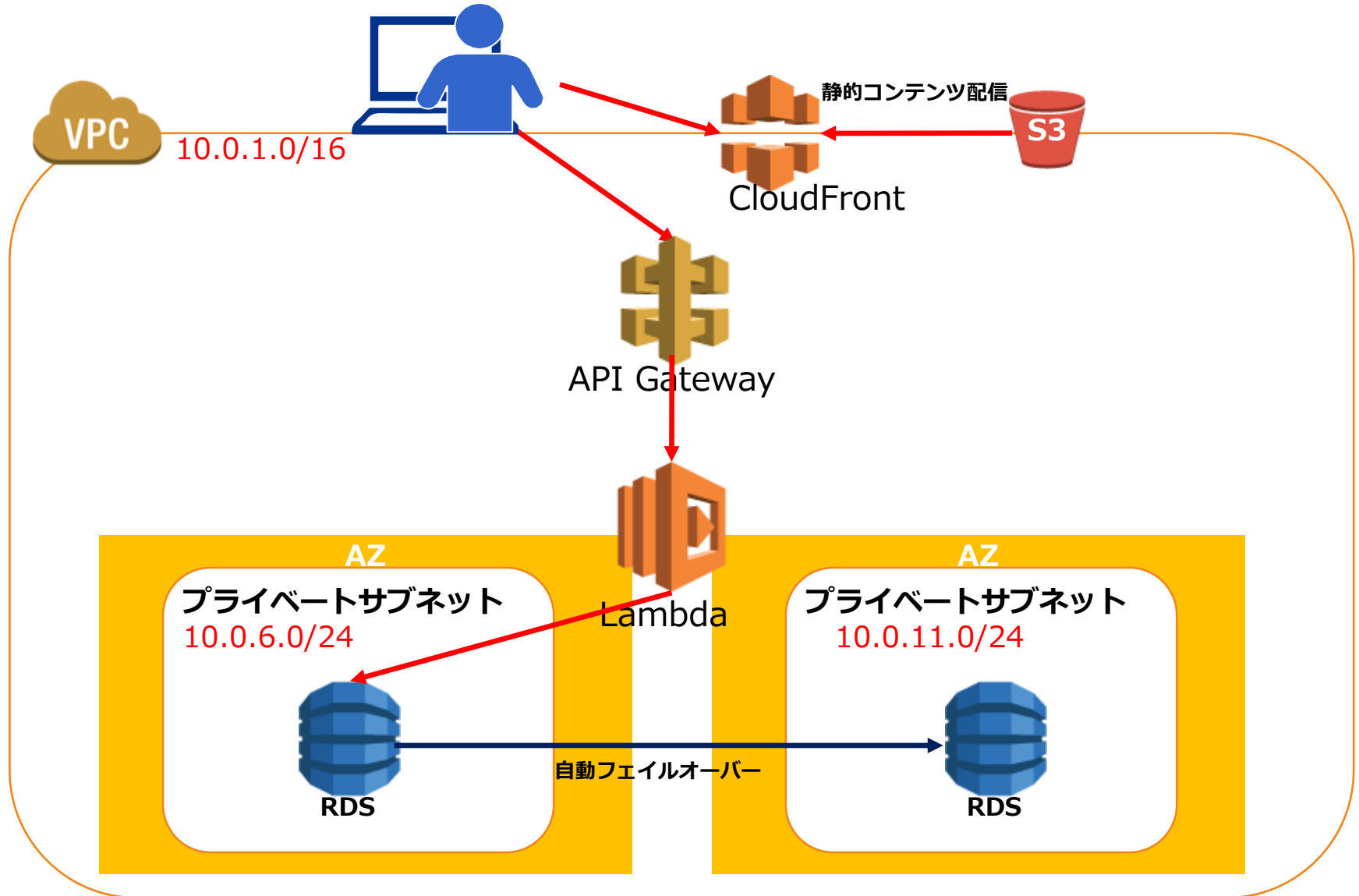
ユースケース

EC2中心のWEBアプリケーションの構築をサーバレス化する



ユースケース

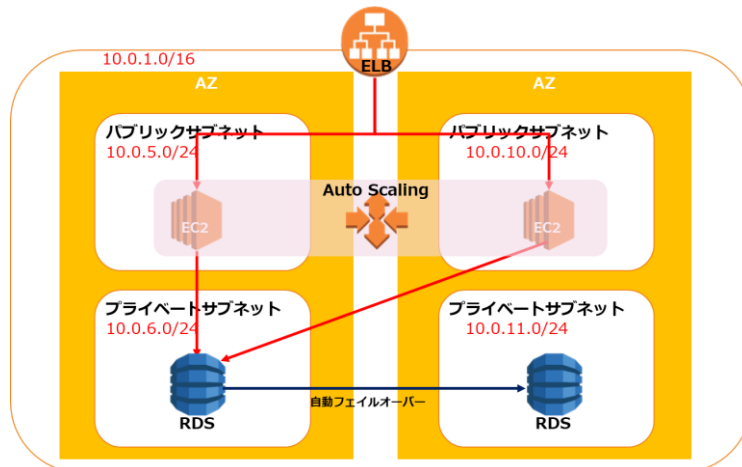
EC2中心のWEBアプリケーションの構築をサーバレス化する



AWSにおけるAPI提供

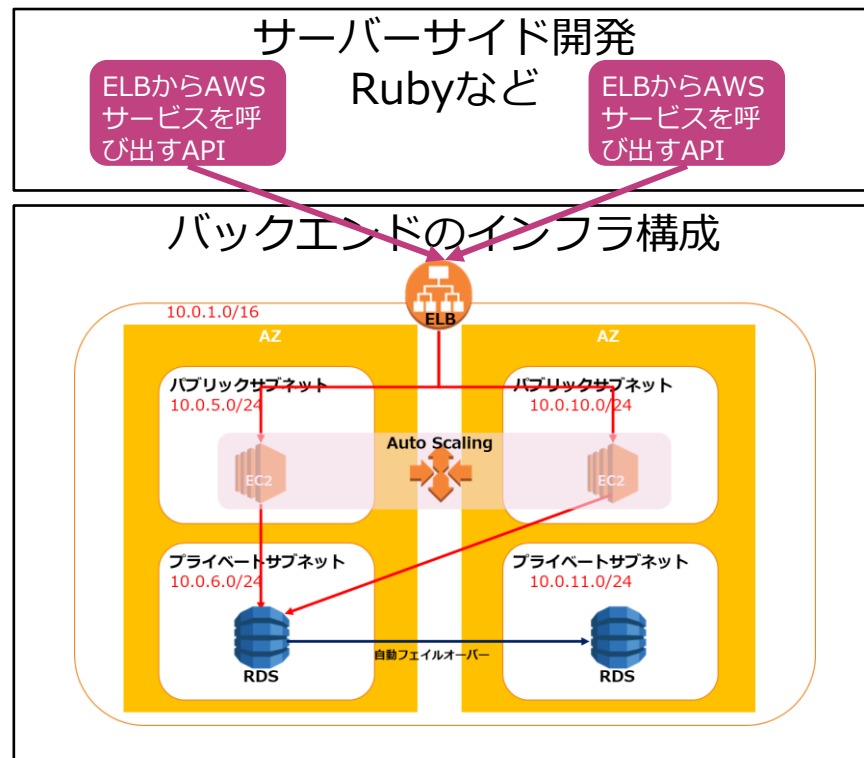
一般的なアプリケーションのサーバーサイド開発では、ELB経由でAPIを提供する。

バックエンドのインフラ構成



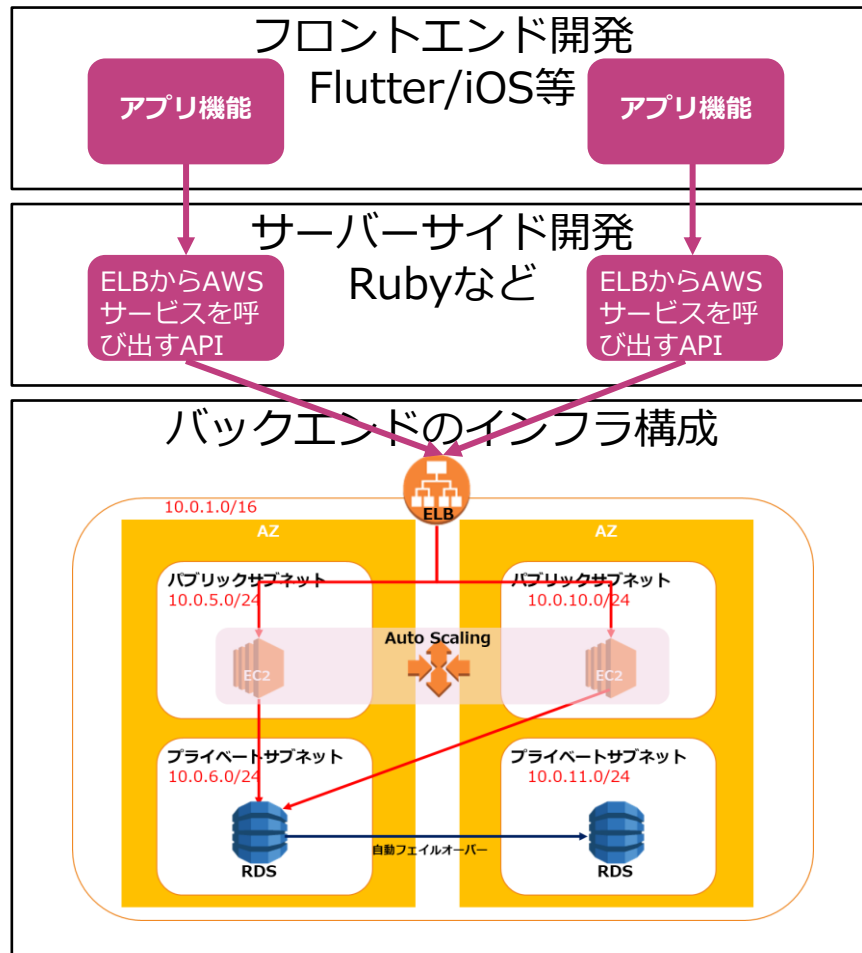
AWSにおけるAPI提供

一般的なアプリケーションのサーバーサイド開発では、ELB経由でAPIを提供する。



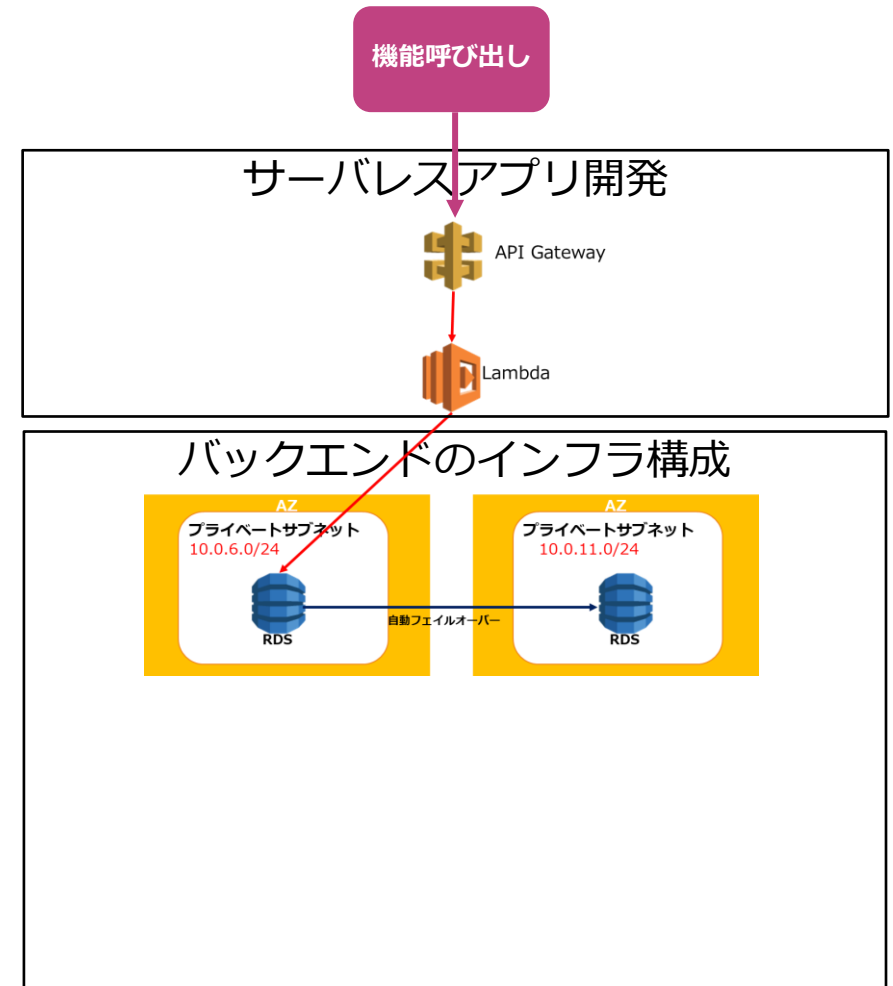
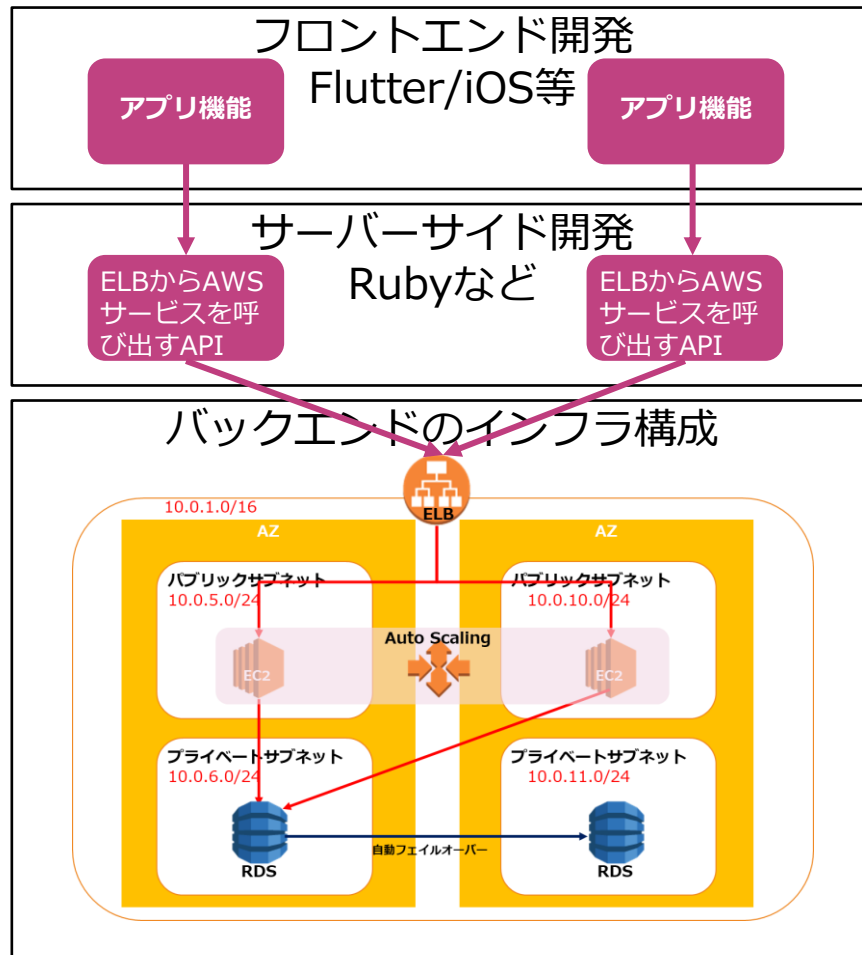
AWSにおけるAPI提供

一般的なアプリケーションのサーバーサイド開発では、ELB経由でAPIを提供する。



AWSにおけるAPI提供

API GatewayはLambda関数を主とした簡易アプリケーション用のAPI提供に特化



APIのタイプ

実現したい通信方式に応じて利用するAPIタイプを選択する。

HTTP API	REST API よりも低いレイテンシーとコストで RESTful API を作成する AWS Lambda 関数またはルーティング可能な HTTP エンドポイントにリクエストを送信できる
Restful API	バックエンドの HTTP エンドポイント、Lambda 関数、その他の AWS のサービスを使用する。主に同期通信に依存するアプリケーションに利用する。 REST APIはクライアントがサービスにリクエストを送信し、サービスが同期的に応答するリクエスト/レスポンスモデルを使用する。
WebSocket API	双方向用の通信方式でクライアントはサービスにメッセージを送信し、サービスは個別にクライアントにメッセージを送信することで豊かなクライアント/サービスの対話を実現する。 チャットアプリ、コラボレーションプラットフォーム、マルチプレイヤーゲーム、金融取引プラットフォームなどのリアルタイムアプリに使用

API Gatewayの料金

API Gatewayは選択したAPIタイプに応じて料金形式が異なる。

HTTP API	使用した API コールの分だけ料金が発生
Restful API	受信した API コールと、転送データ量に対してのみ料金が発生
WebSocket API	受送信したメッセージ数および分単位の接続合計時間から料金が発生

API Gatewayの統合

API GatewayはAPIメソッドを作成して、Lambda関数やWEBサイトなどのバックエンドエンドポイントと統合する。

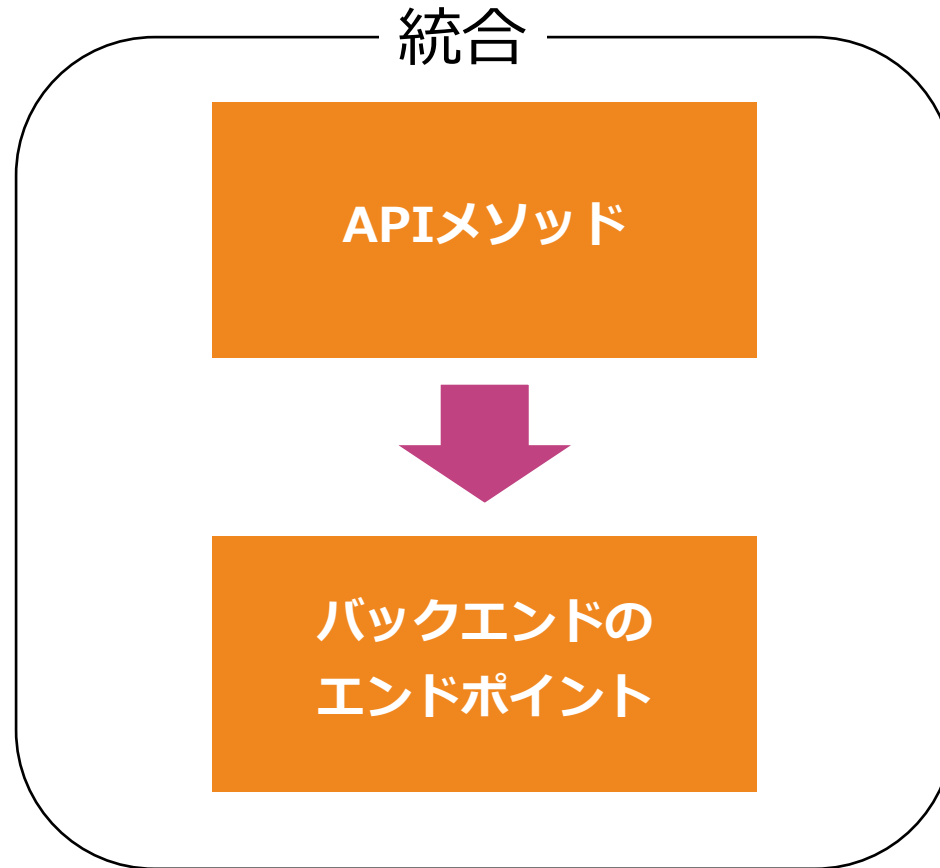


APIメソッド

バックエンドの
エンドポイント

API Gatewayの統合

API GatewayはAPIメソッドを作成して、Lambda関数やWEBサイトなどのバックエンドエンドポイントと統合する。



統合タイプ

API Gatewayは4つの統合タイプを利用してエンドポイントを統合する。

Lambda統合	Lambda関数との統合方式。Lambda関数はAPI Gatewayと密接に連携しており、プロキシ統合またはLambda 非プロキシ (カスタム) 統合を使用して、API メソッドを Lambda 関数に統合する。
HTTP統合	バックエンドのHTTPエンドポイントを公開する。 HTTP プロキシ統合または HTTP カスタム統合を使用して、API メソッドを HTTP エンドポイントに統合する。 80、443、および 1024-65535のエンドポイントポートをサポートする。
プライベート統合	VPC外のクライアントからVPC内にあるHTTP/HTTPSリソースにアクセスするために利用する統合方式。 API Gateway がサポートする認証方法よりAPIへのアクセスを制御できる。
Mock統合	バックエンドを統合することなく、API Gateway から直接 API レスポンスを生成できる統合方式。 これによりAPI を操作する必要がある他の依存チームのブロックを解除できる。 また、API の概要や API へのナビゲーションを提供できる API のランディングページをプロビジョニングすることが可能となる。

API GatewayのLambda統合

API GatewayをLambda関数と統合する際はプロキシ統合または非プロキシ（カスタム）統合から選択する。

Lambda プロキシ統合

- Lambda プロキシ統合では、API Gateway がクライアントリクエスト全体をバックエンド Lambda 関数にマッピングして、Lambda関数と統合する。API Gatewayに許可ロールを設定して統合設定するだけで**自動的にマッピングして統合できる**。
- クライアントが API リクエストを送信すると、API Gateway は、統合された Lambda 関数に raw リクエストをそのまま渡す。リクエストパラメータの順序は保持されない。
- このリクエストデータには、リクエストヘッダー、クエリ文字列パラメータ、URL パス変数、ペイロード、および API 設定データが含まれる。
- 設定データには、現在のデプロイステージ名、ステージ変数、ユーザー ID、または承認コンテキスト (存在する場合) を含める。

Lambda 非プロキシ統合 (カスタム統合)

- プロキシ統合のセットアップステップに加えて、受信リクエストデータがどのように統合リクエストにマッピングされるか、統合レスポンスデータの結果がメソッドレスポンスに**マッピングされるかを指定する**。

APIエンドポイントのタイプ

API トラフィックの発信元に応じて、エッジ最適化、リージョン別、またはプライベートを選択する。

エッジ最適化 API エンドポイント

CloudFrontと連携してグローバルにクライアントが分散している場合に最適にルーティングする。API リクエストは最寄りの CloudFront POP (Point Of Presence) にルーティングされる。

CloudFront は、リクエストをオリジンに転送する前に、Cookie 名の自然な順序で HTTP Cookie を並べ替える。

リージョナル API エンドポイント

同じリージョンのクライアントに最適なルーティングをする。同じリージョンの API を呼び出す場合などにリージョン別 API は接続のオーバーヘッドを減らすことができる。

CloudFrontを利用することも可能である。

プライベート API エンドポイント

VPC内のクライアントに最適なルーティングをする。VPCからしかアクセスできないAPI エンドポイント。インターフェイス VPC エンドポイントにはVPC 内に作成するエンドポイントネットワークインターフェイス (ENI) を利用する。

リソースポリシーを利用してアクセスを制御できる。

キャッシュ機能の利用

キャッシュを有効にすると、エンドポイントへの呼び出しの数を減らし、API へのリクエストのレイテンシーを短くできる



デフォルトの TTL 値は 300 秒
最大 TTL 値は 3600 秒
TTL=0 は、キャッシュを無効化

スロットリングの利用

リクエスト数が多すぎる場合、制限をかけることで、トラフィックの急増に対してバックエンドサービスを守る。

サーバー側のスロットリング制限

全てのクライアントに対するリクエストを制限する。全体のリクエストが多すぎるためにバックエンドサービスが処理しきれなくなることを防ぐことができる。

クライアントあたりのスロットリング制限

クライアントごとに「使用量プラン」に応じて制限を行う。
特定のユーザーからのリクエストが多い場合に有効

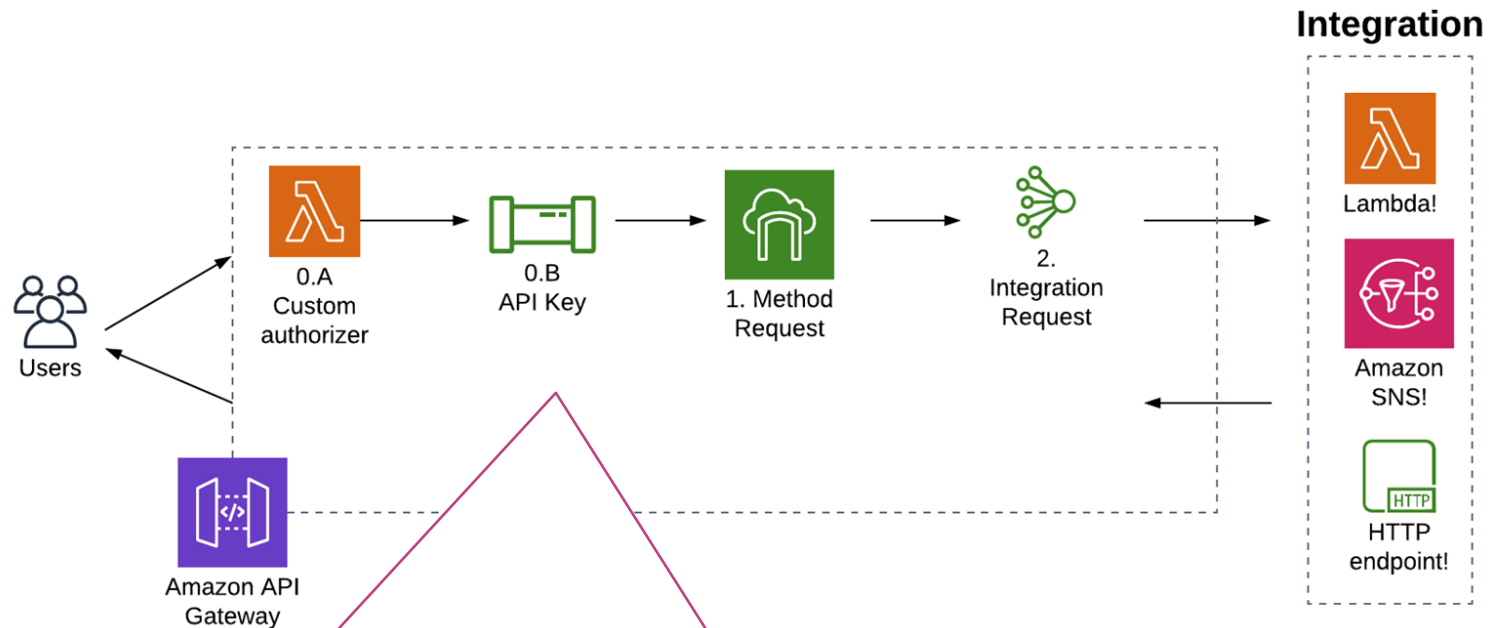
API Gatewayの認証方式

API Gatewayへのアクセス認証には様々なタイプを利用可能

リソースポリシー (REST APIのみ)	JSON形式のリソースポリシーを定義することで、他のリソースからのAPI Gatewayへのアクションの許可または拒否を設定する。
IAM認証	APIのアクセス権限を設定したIAMポリシーを作成し、IAMユーザやIAMロールに付与してAPIへのアクセスを制御する。 APIメソッドでIAM認証を有効化する
Lambda オーソライザー	Lambda関数を作成することで、認証プロバイダーでの認証結果を元に、APIへのアクセス制御をメソッド単位で実施
Cognito オーソライザー	認証プロバイダとしてCognitoユーザープールを用いて、APIへのアクセス制御をメソッド単位で実施

使用量プランとAPIキー

使用量プランとAPIキーを利用して、アクセスするユーザーやアクセス回数などを制御できる。



- ❑ 使用量プランにより、APIキーにアクセス可能回数となるクォータ、アクセス頻度となるレート、バーストを設定できる。
- ❑ 使用量プランにより、APIの使用回数を取得できる。
- ❑ API キーを使用して、API クライアントとユーザーを識別して、APIキーを持つユーザのみアクセス可能なAPIを作成できる。
- ❑ API キーと Lambda オーソライザー、IAM ロール、または Amazon Cognito を一緒に使用して API へのアクセスを制御できる。